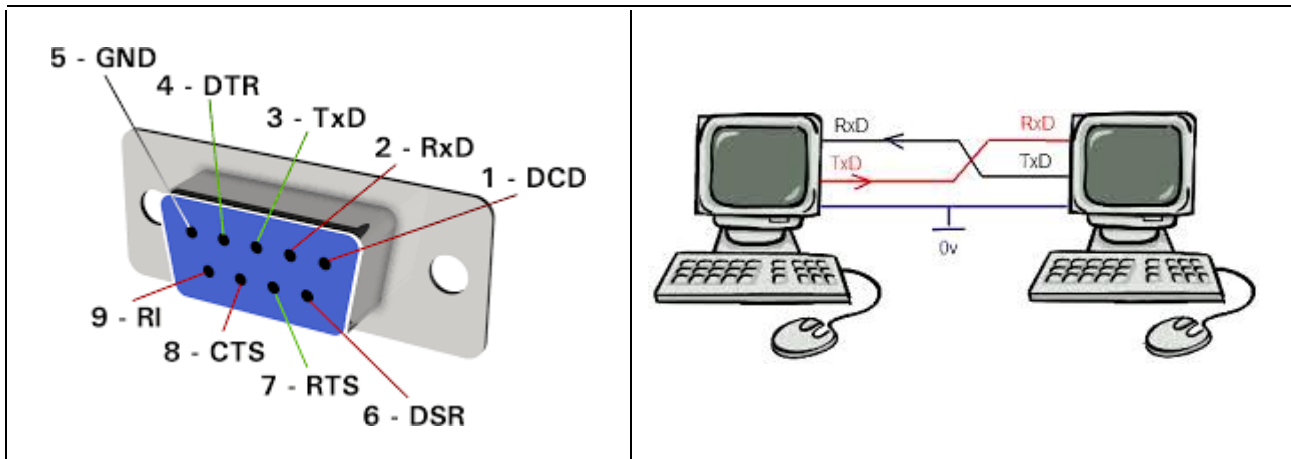


Chapitre 3:

Communication par transmission série RS232



Objectifs du cours :

Ce cours traitera essentiellement les points suivants :

- Définitions : ETTD, ETCD, liaison série, transmission série et parallèle, synchrone et asynchrone
- Nature des liaisons de données : simplex, half-duplex et full-duplex
- La jonction
- Signal analogique et signal numérique (vitesse de transmission, types de transmission et types de liaison)
- Transmission série (câblage, circuit UART et USART, transmission asynchrone, port série RS-232, port série virtuel)
- Exercices d'application

Si on considère les notions de canal de transmission et de support de transmission, pour définir leurs caractéristiques physiques et leurs performances, il faut s'intéresser aux configurations possibles des liaisons de données ainsi qu'aux équipements nécessaires à la transmission des données.

DÉFINITIONS

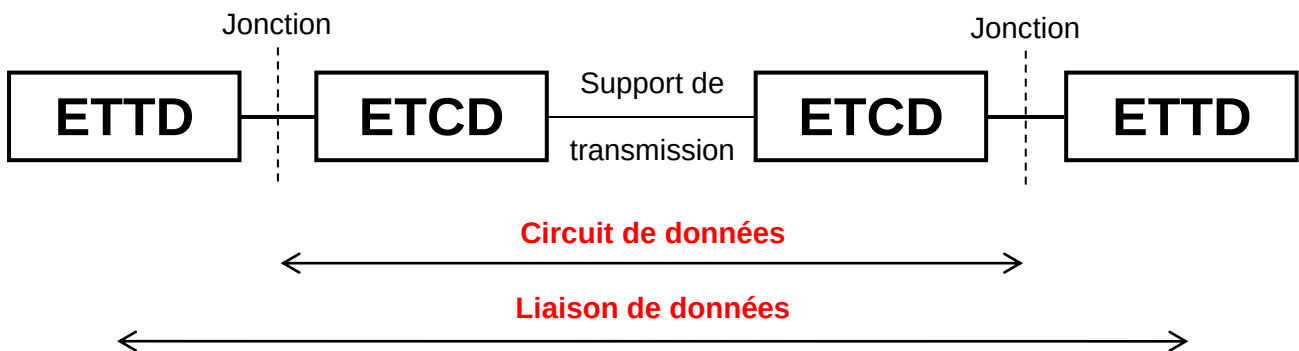
La communication entre deux équipements informatiques réalise une liaison constituée des éléments suivants :

Deux **ETTD** (**É**quipement **T**erminal de **T**raitement de **D**onnées), l'un à chaque extrémité de la liaison. Ces équipements génèrent les données ; par exemple un ordinateur.

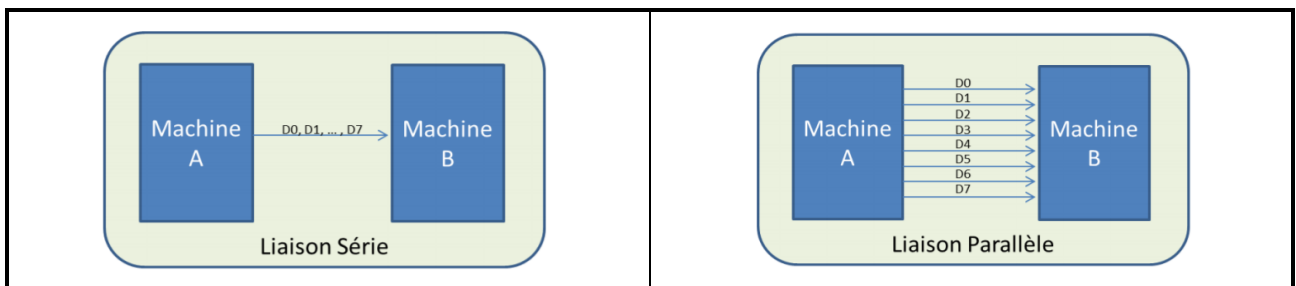
Une ligne de transmission.

Deux **ETCD** (**É**quipement **T**erminal d'un **C**ircuit de **D**onnées) qui adaptent les données issues de l'ETTD au support de transmission (modulation, codage) et gèrent la liaison (établissement, maintien et libération de la ligne) ; par exemple un modem.

La figure ci-dessous représente **une liaison de données**. On appelle « liaison de données » l'ensemble des éléments matériels et logiciels réalisant les fonctions nécessaires à l'acheminement des données. La liaison gère le circuit de données et s'occupe de la correction et de la détection des erreurs. L'interface entre l'ETTD et l'ETCD, ou **jonction**, permet à l'ETTD de contrôler le circuit de données (établissement et libération, initialisation de la transmission, etc).



On appelle « **liaison série** » une liaison dans laquelle les bits issus de l'ETTD sont émis l'un après l'autre sur le support. C'est le mode de transmission utilisé sur une liaison de données. À l'intérieur de l'ETTD en revanche, les données circulent en parallèle, c'est-à-dire sur plusieurs « fils » ; si n « fils » sont disponibles, n bits sont transmis simultanément. Ce type de transmission permet des débits élevés mais est efficace sur des courtes distances seulement. En effet le phénomène de diaphonies, c'est-à-dire le rayonnement électromagnétique du signal d'un fil sur les autres, perturbe les communications.



La transmission série est privilégiée dans le cadre des réseaux.

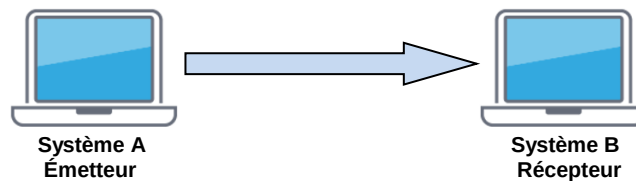
Il est donc nécessaire de réaliser une conversion parallèle/série lors de la transmission de données issues d'un ordinateur. Les données sur la liaison série sont émises au rythme de l'horloge de l'émetteur. Pour lire les données, le récepteur doit connaître la fréquence de l'horloge émettrice, ce qui peut être réalisé de deux façons différentes :

- **Transmission synchrone**, le signal d'horloge est transmis par l'émetteur. Il peut être transporté par un fil, reconstitué à partir du spectre du signal de données, ou encore reconstitué à partir de caractères de synchronisation insérés au début des trames.

- **Transmission asynchrone**, le récepteur possède une horloge interne qu'il doit synchroniser sur la séquence de bits reçue.

NATURE DES LIAISONS DE DONNÉES

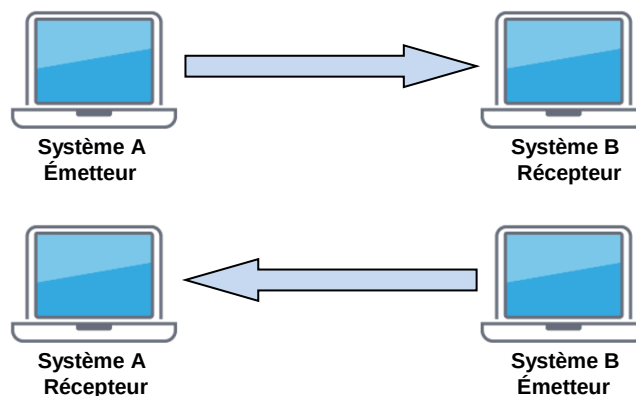
SIMPLEX (UNIDIRECTIONNELLE)



Dans ce type de liaison, un seul sens de transmission est possible.

Exemples : radiodiffusion, télédiffusion.

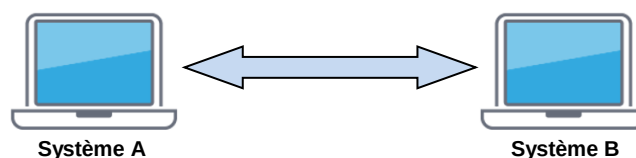
HALF-DUPLEX (BIDIRECTIONNELLE ALTERNÉE)



Dans ce type de liaison, les deux sens de transmission sont possibles, mais alternativement. Ce type de liaison peut être nommé « semi-duplex ».

Exemples : Talkie-walkie, l'ethernet sur bus.

FULL-DUPLEX (BIDIRECTIONNELLE SIMULTANÉE)



Dans ce type de liaison, les deux sens de transmission sont assurés simultanément.

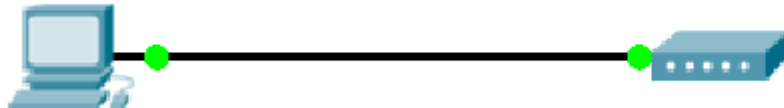
Exemples : Téléphone, Internet.

LES JONCTIONS

Pour assurer une transmission de données, il est nécessaire d'échanger des informations (données de supervision, commandes, rythmes d'horloge,...) entre l'ETCD et l'ETTD.

Jonction

V24, V28, ISO 2110, ISO 4902



V24 : nom usuel de la jonction, il définit l'aspect fonctionnel de la jonction.

V28 : définit les tensions électriques de la jonction (0 logique : $+3\text{ V} \leq U \leq +25\text{ V}$; 1 logique : $-25\text{ V} \leq U \leq -3\text{ V}$).

ISO 2110 : définit l'aspect mécanique et connectique de la jonction (25 broches).

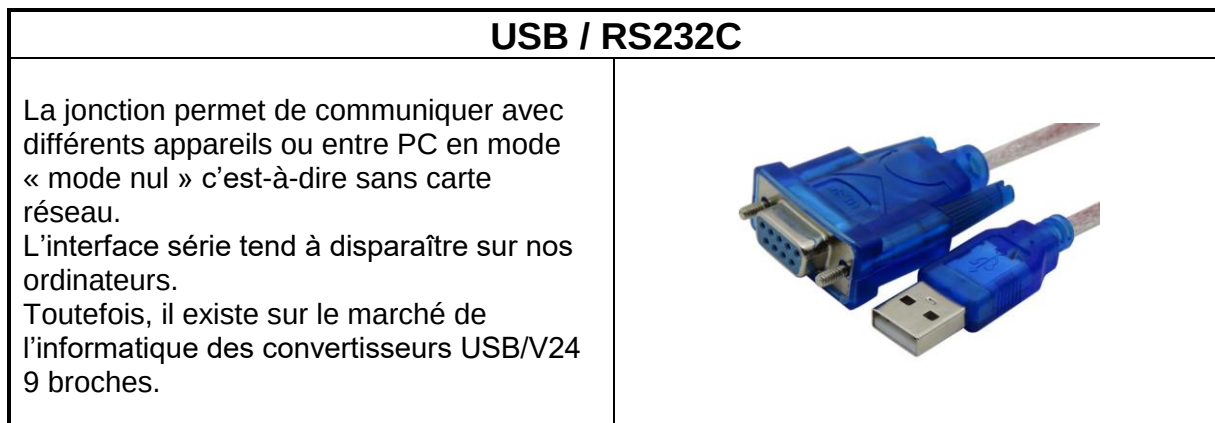
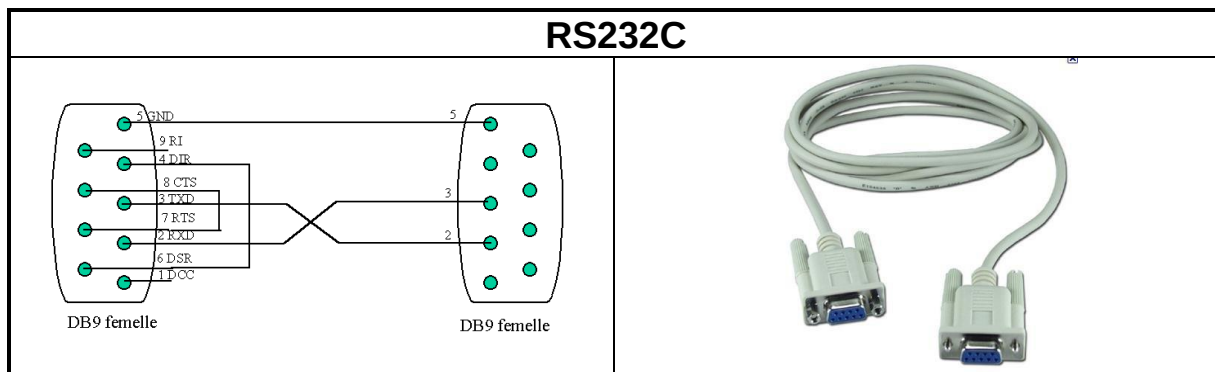
ISO 4902 : définit l'aspect mécanique et connectique de la jonction (9 broches).

Pour les U.S., cette jonction est nommée « **RS232C** ».



D'ordinaire **des niveaux de +12 V et -12 V** sont utilisées.

La norme V28 indique qu'un 1 est reconnu si la tension est inférieure à -3 V et un 0 est reconnu si la tension est supérieure à +3 V.



Le standard RS232 prévoit des formats de transmission synchrone et asynchrone. La transmission asynchrone est beaucoup plus démocratisée en raison de sa souplesse d'utilisation.

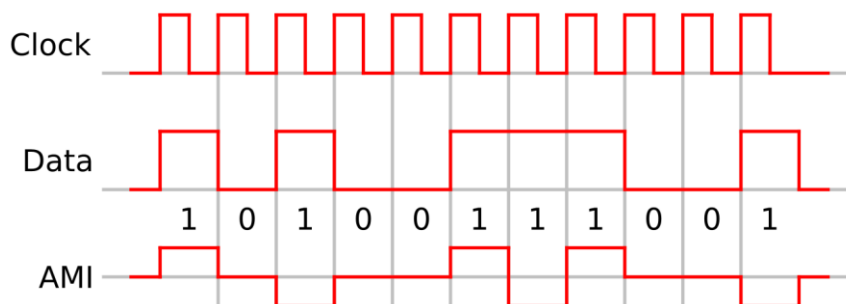
SIGNAL ANALOGIQUE ET SIGNAL NUMÉRIQUE

On distingue 2 types de signaux :

- **analogique** en utilisant un modem (pour modulateur-démodulateur) qui est un périphérique servant à convertir les données numériques de l'ordinateur en signal modulé, dit « analogique », transmissible par un réseau analogique.

- **numérique** (bande de base ou « baseband »), les données numériques sont transmises après un codage, où la durée de chaque bit est constante (nominal bit-time), pour s'adapter aux caractéristiques de la ligne de transmission.

Dans une transmission numérique, les n bits sont transmis au rythme d'une horloge (clock) dont la valeur de la période permet de définir la notion de **débit** (vitesse de transmission) : nombre de bits transmis par unité de temps.



Exemple de codage d'une transmission numérique



Codage AMI (Alternate Mark Inversion) : codage à 3 niveaux (ternaire)

VITESSE DE TRANSMISSION

On distingue :

- **le baud** (Bd) qui est l'unité de mesure du nombre de symboles transmissibles par seconde. Dans le cas d'un signal modulé utilisé dans le domaine des télécommunications, le baud est l'unité de mesure de la rapidité (vitesse) de modulation.

- **le bit par seconde** (bit/s) est l'unité de mesure du nombre d'informations effectivement transmises par seconde (vitesse de transmission).

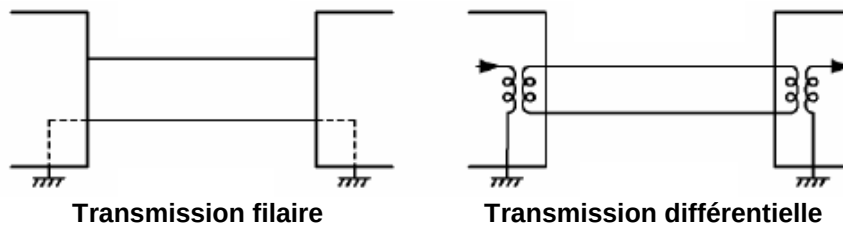


Il ne faut pas confondre le baud avec le bit par seconde car il est souvent possible de transmettre plusieurs bits par symbole. Cependant, dans le cas où le signal n'est pas modulé : la vitesse de modulation = la vitesse de transmission, on dit que l'on est en « **bande de base** ».

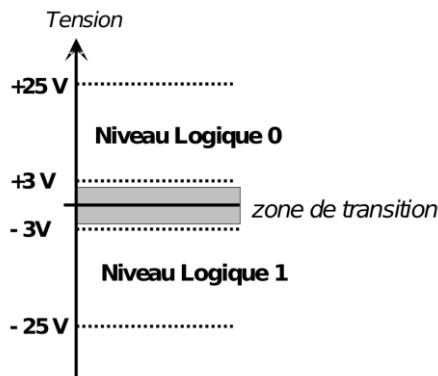
TYPES DE TRANSMISSION

On distingue deux types de transmission pour un signal numérique :

- **la transmission filaire** (ou unipolaire) où le signal est transmis sur un fil par rapport à une tension de référence de 0V (la masse). Elle est utilisée par exemple pour la liaison série RS-232.
- **la transmission différentielle** est une méthode de transmission de signaux sur une ligne symétrique, c'est-à-dire utilisant deux conducteurs différents (une paire) pour transmettre l'information.



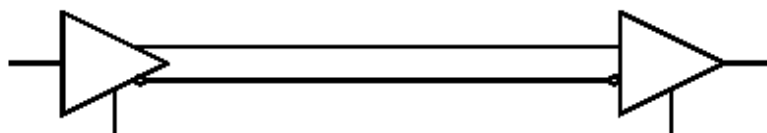
Transmission filaire (ou unipolaire) :



Codage des niveaux 0 et 1 pour la liaison série RS-232

Transmission différentielle :

La transmission différentielle consiste à envoyer sur un fil le signal et sur l'autre fil le signal opposé. On reconstitue le signal à l'arrivée en effectuant la différence des signaux.



Même si une perturbation électromagnétique dégrade le signal d'un fil, la différence est inchangée, car on suppose que cette perturbation dégrade de la même manière l'autre fil. On réalise ainsi une transmission avec une meilleure immunité au bruit.

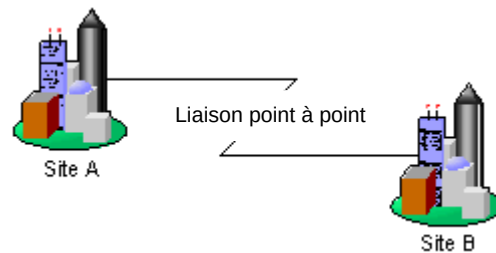


Cette méthode est notamment utilisée par les couches matérielles RS-422 et RS-485, ainsi que par les bus informatiques USB, SATA, FireWire.

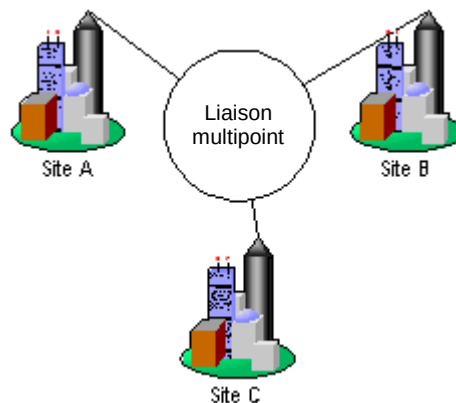
TYPES DE LIAISON

On distingue deux types de liaison :

- **la liaison point à point** (2 participants) : utilise un dialogue full-duplex (le plus courant) ou half-duplex (plus lent). On utilise une topologie libre avec différentes possibilités de câblage (arborescente, étoile, maillage, ...).



- **la liaison multipoint** (n participants) : utilise un dialogue half-duplex (le plus courant) ou full-duplex (alternative possible). On utilise soit une topologie en bus (avec un câblage en chaîne, un câblage avec des connexions en T, un câblage avec des prise vampires ou un câblage en étoile avec un concentrateur/hub), soit une topologie en anneau (bus fermé).



TRANSMISSION SÉRIE

La transmission série domine dès que les composants ou périphériques à relier sont à « quelque distance ». L'ensemble des télécommunications s'établit sur des liaisons « série ».

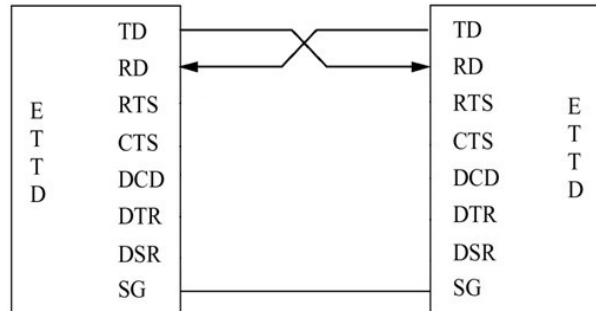
La transmission série est très présente dans le monde industriel :

- pour relier des capteurs/actionneurs (sensor bus) ou des composants de bas niveau, on utilise des technologies comme le bus **I2C**, **1-Wire**, **ASi**, ...
- pour relier des périphériques (appareils divers, système de commande,...), on utilise des bus de terrain (field bus) comme le **bus CAN**, **DMX**, les liaisons **RS-232**, **RS-422** ou **RS-485**, ...

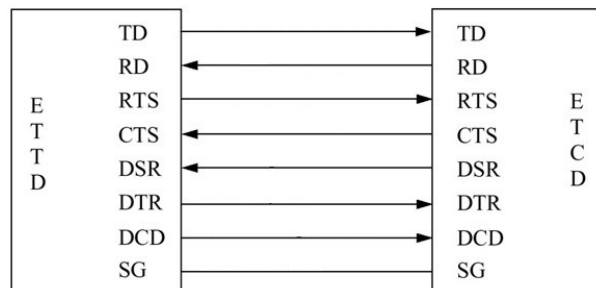
La transmission série se fait généralement par **trames** et nécessite la mise en oeuvre de **protocoles**.

CÂBLAGE

Liaison « croisée » entre 2 ETTD ou 2 ETCD (ici une liaison minimum à 3 fils) :



Une liaison « droite » entre ETTD et ETCD :



CIRCUIT UART ET USART

La transmission série est assurée par un circuit électronique qui permet de mettre en série (sérialiser) les octets à transmettre. Cela signifie que les bits constituant l'octet sont transmis les uns après les autres. On distingue :

- **UART** (Universal Asynchronous Receiver Transmitter) est le composant utilisé pour réaliser une transmission asynchrone.
- **USART** (Universal Synchronous & Asynchronous Receiver Transmitter) est un circuit électronique permettant de communiquer de manière synchrone.



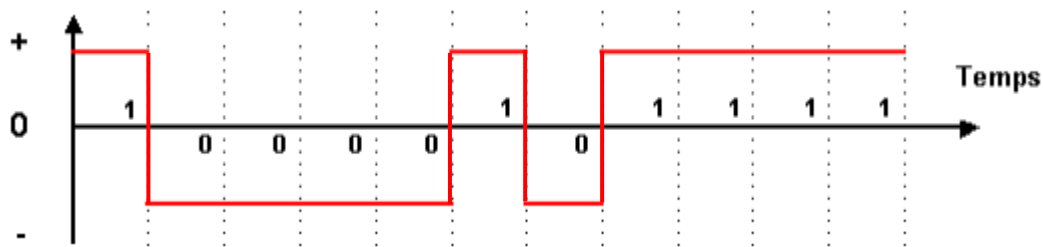
Pour ces circuits, les vitesses de transmission sont normalisées par des multiples ou des sous-multiples de 9600 bauds, ici l'unité baud correspondant à un bit par seconde.

TRANSMISSION ASYNCHRONE

Dans une transmission asynchrone, seules les données sont transmises au récepteur. Chaque équipement utilise son horloge pour traiter les bits transmis.

Exemple du codage **NRZ** (Non-Return-to-Zero) : ici, le bit 1 est représenté par une

tension positive, et le bit 0 par une tension négative. Il n'existe pas d'état intermédiaire.



Codage NRZ

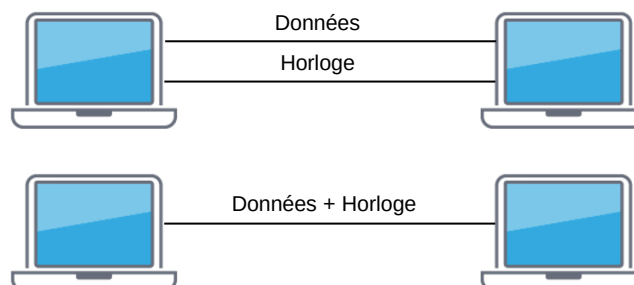
On remarque qu'il n'y a pas de transition générée lors d'une longue séquence de 0 (ou de 1), ce qui rend la synchronisation difficile, voire impossible. Combien de 0 ou combien de 1 seront lus ?



Cette transmission limite le nombre de bits transmis (généralement un octet ou un caractère).

TRANSMISSION SYNCHRONES

La transmission synchrone doit assurer la transmission des données ainsi que l'horloge de synchronisation nécessaire à leur décodage.



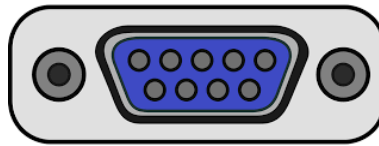
La transmission synchrone permet de transmettre des séquences de bits plus longues (n octets) appelées **frame**.

Le codage Manchester est un codage synchrone utilisé sur les réseaux Ethernet à 10 Mbits/s (10BASE5, 10BASE2, 10BASE-T, 10BASE-FL).

LE PORT SÉRIE RS-232

RS-232 est une norme standardisant une voie de communication de type série sur trois fils minimum. Disponible sur presque tous les PC depuis 1981 jusqu'au milieu des années 2000, il est communément appelé le « **port série** ».

Sur les systèmes d'exploitation Windows, les ports RS-232 sont désignés par les noms COM1, COM2, etc. Cela leur a valu le surnom de « **ports COM** », encore utilisé de nos jours.



Le port « COM »

Brochage :

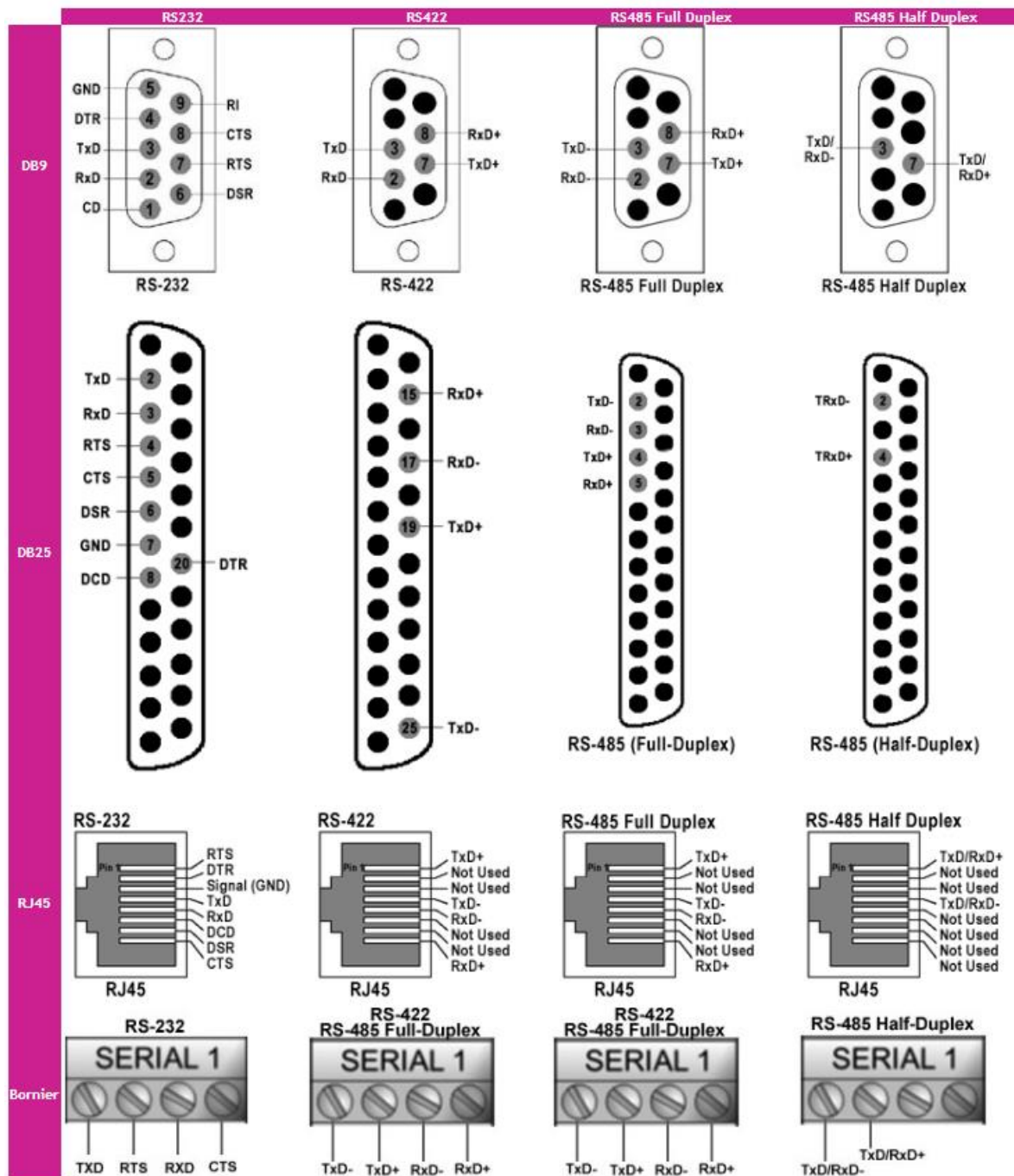
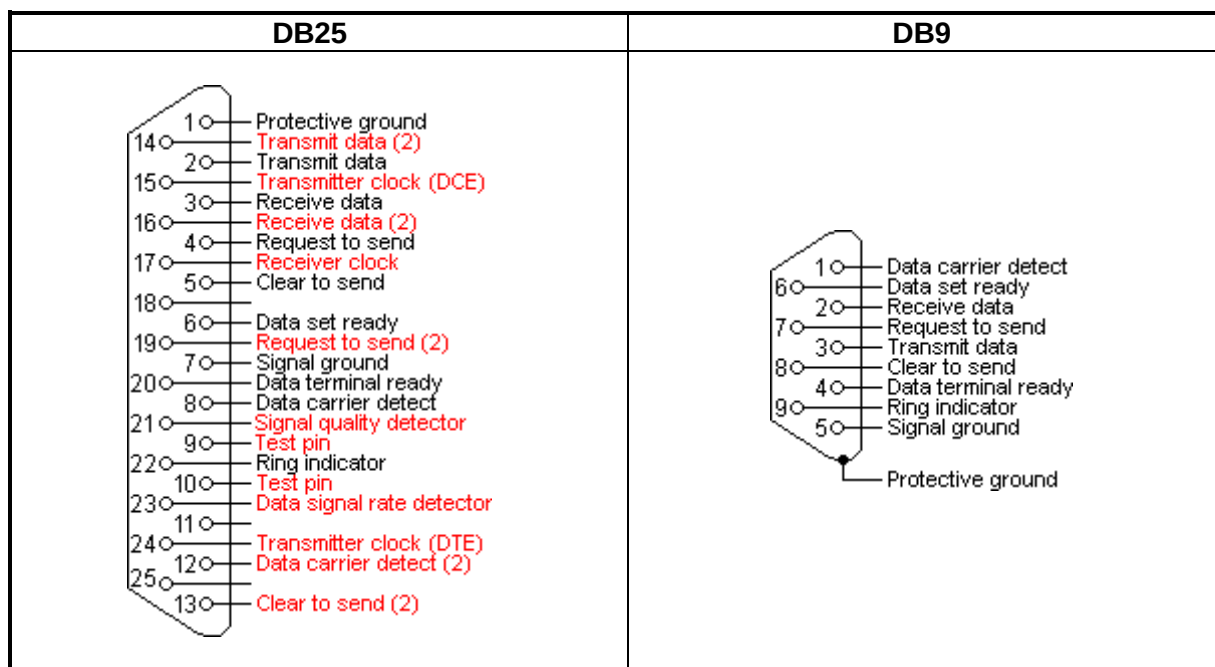
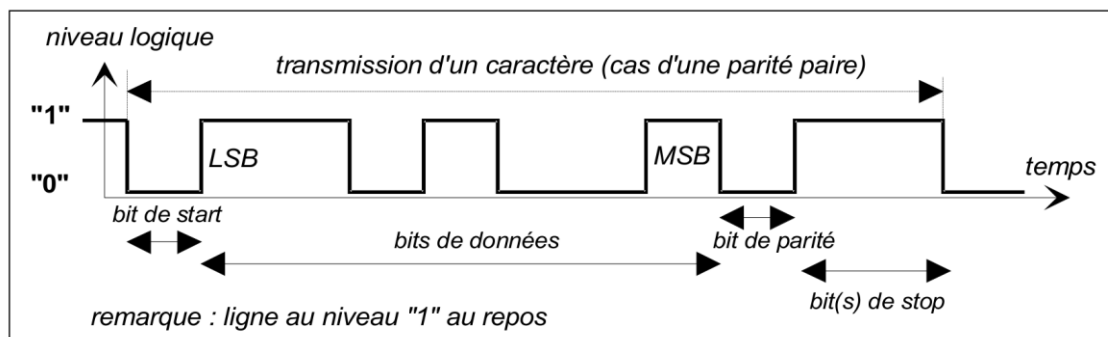


Schéma de câblage RS-232/422/485 pour les connecteurs DB9 et RJ45

Repérage et identification des broches :



Trame série UART :



Bit de START : le front descendant indique le début de la trame ce qui doit permettre au récepteur de se synchroniser ;

Bits de DONNÉES dont le nombre varie normalement entre 5 et 8, en fonction du codage du caractère (ASCII, ...) ;

Bit de PARITÉ (facultatif) : bit généré lors de l'émission et testé lors de la réception pour détecter une erreur de transmission. La parité est paire (even) lorsque le nombre de bits (donnée + parité) est pair. La parité est impaire (odd) lorsque le nombre de bits (donnée + parité) est impair ;

Bit(s) de STOP : durée (1, 1.5 ou 2 bit-time) inter-trame pour dissocier la fin d'émission du caractère courant du début du caractère suivant (bit de start).

Détection d'erreur :

La détection d'erreur de transmission est basée sur l'utilisation d'une somme de contrôle (**checksum**). La somme de contrôle (checksum) est un nombre qu'on ajoute à un message à transmettre pour permettre au récepteur de vérifier que le message reçu est bien celui qui a été envoyé.

Par exemple :

- le bit de parité d'une trame UART ;
- le Contrôle de Redondance Cyclique (Cyclical Redundancy Check) est utilisé dans les trames pour détecter une erreur de transmission. Il est généralement codé sur 16 bits ou 32 bits. Le CRC est calculé par l'émetteur avant d'être transmis. Le récepteur calcule aussi un CRC avec la trame reçue et le compare avec le CRC reçu : des valeurs différentes indiqueront une erreur dans la transmission du message. Dans les systèmes industriels, le calcul du CRC est souvent basé sur l'utilisation du OU EXCLUSIF (XOR).

Contrôle de flux

Le contrôle de flux, dans un réseau informatique, représente un asservissement du débit binaire des données transmises de l'émetteur vers le récepteur. Le « stop and wait » est la forme la plus simple de contrôle de flux. En communication série asynchrone RS-232, deux modes de contrôle de flux sont proposés :

- **matériel** (hardware) via les lignes **RTS/CTS** (Ready To Send/Clear To Send). Lorsque le buffer de réception est plein, RTS est désactivé. Il sera réactivé lorsque les données du buffer auront été lues (DTR « Data Terminal Ready » et DSR « Data Set Ready » peuvent être utilisés selon le même principe). L'émetteur doit donc scruter son CTS pour savoir s'il peut émettre ou non. Le contrôle de flux hardware en RS-232 nécessite 5 fils (Rx, Tx, Gnd, RTS, CTS).
- **logiciel** (software) via les caractères ASCII **XON/XOFF**. Ce protocole basé sur le même principe que le précédent si ce n'est que Xon et Xoff sont des caractères qui valent respectivement 0x10 et 0x13 en hexadécimal. Le récepteur signale que son buffer de réception est plein en envoyant un caractère Xoff à l'émetteur. Lorsqu'il peut à nouveau accepter des caractères en réception, il envoie Xon. Le contrôle de flux logiciel ne nécessite que 3 fils (Rx, Tx, Gnd).

Les normes RS-232, RS-422 et RS-485

Spécifications	RS 232	RS 422	RS 485
Type de communication	Unipolaire	Différentiel	Différentiel
Connexions électriques minimales	3 fils Tx, Rx et masse	5 fils Paire Tx, Paire Rx et masse	3 fils Paire Tx/Rx, et masse
Nombre de transmetteurs et récepteurs alloués par la ligne	1 transmetteur 1 récepteur	1 transmetteur 31 récepteurs	32 transmetteurs 32 récepteurs
Longueur maximum de câble	16,5 m	1320 m	1320 m
Débit maximum	64 Kbits/s	10 Mbits/s	10 Mbits/s

LE PORT SÉRIE VIRTUEL

On utilise maintenant des adaptateurs USB/RS-232 car les PC ne disposent plus d'interfaces physiques RS-232. Cela revient à exploiter un **port série virtuel**.

Un port série virtuel est une solution logicielle qui émule un port série standard. Cela permet généralement :

- d'augmenter le nombre de ports série (sans installation de matériel supplémentaire mais dans les limites des ressources disponibles) ;
- de partager les données en provenance d'un périphérique entre plusieurs applications ;
- de raccorder un périphérique série standard (RS232, ...) sur un port USB avec un adaptateur (manque ou absence de ports série physiques disponibles).

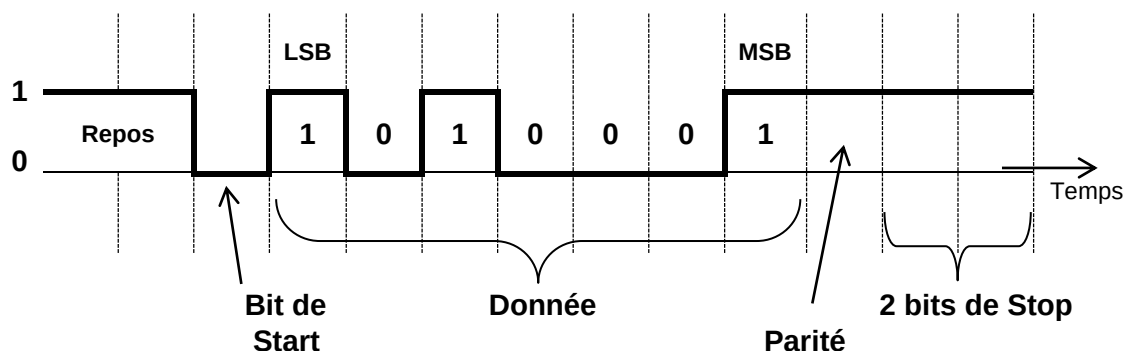
QUELQUES EXEMPLES

Exemple d'une trame logique d'une liaison série asynchrone :

On souhaite transmettre la donnée « **0x45** » soit le caractère « **E** » avec les conditions suivantes :

- Donnée sur 7 bits
- Parité paire
- 2 bits de stop

0x45 = %1000101



Remarques :

Le poids faible (LSB) de la donnée est transmis en premier.

Le récepteur détecte la trame grâce au premier front descendant apparaissant après un état de repos de la ligne. Ce front correspond au début du bit de START. La lecture des bits qui constituent la trame est réalisée à un intervalle de temps régulier définis en fonction de la vitesse de transmission choisie. L'émetteur et le récepteur doivent donc être configurés avec la même vitesse de transmission. On fixe cette valeur en fonction des performances requises à des valeurs généralement conformes aux transmissions standards : 110, 300, 1200, 2400, 4800, 9600, 19200, 38400, 57600, 115200 Bauds (Bd).

Cependant, d'autres vitesses de transmission sont possibles en fonction des UART utilisés.

Le nombre de bits sur une trame asynchrone est généralement limité à 11 :

1 bit de start + 8 bits de données + 1 bit de parité + 1 bit de stop

1 bit de start + 7 bits de données + 1 bit de parité + 2 bits de stop

Exemple d'une trame électrique d'une liaison série asynchrone :

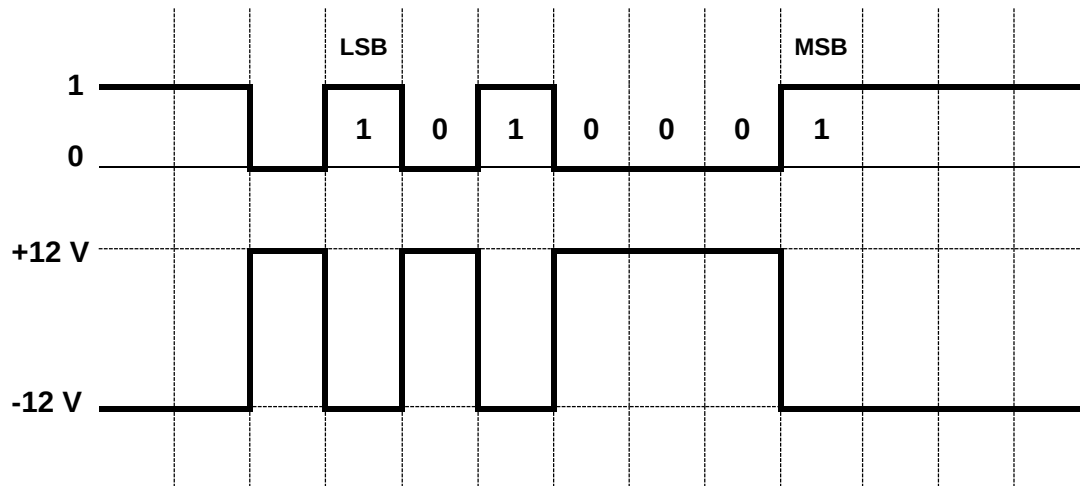
Les niveaux de tension appliqués sur la ligne de transmission sont imposés par des circuits « **pilote de ligne** » et lus au niveau du récepteur par des circuits « **récepteurs de ligne** ».

Les circuits les plus couramment utilisés sont :

Le MC1488 qui est un pilote de ligne,

Le MC1489 qui est un récepteur de ligne,

Le MAX232 qui combine à la fois 2 émetteurs de lignes et 2 récepteurs de ligne,
La norme RS232 prévoit la conversion logique électrique suivante :

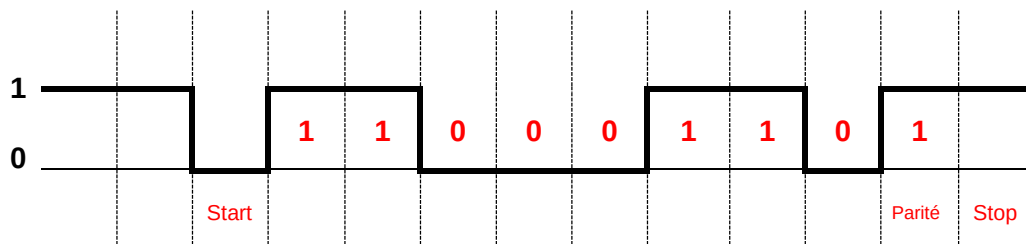


Ci-dessus la correspondance entre la trame logique et la trame électrique pour la même transmission que précédemment (envoi de 0x45).

EXERCICES D'APPLICATION

Question 1

Soit la trame logique d'une liaison série asynchrone ci-dessous.
Sachant que la donnée transmise est codée sur un octet, qu'il y a une parité et 1 bit de stop.
Quel est le type de parité utilisé ?



Question 2

Retrouvez la donnée transmise en hexadécimale.

Question 3

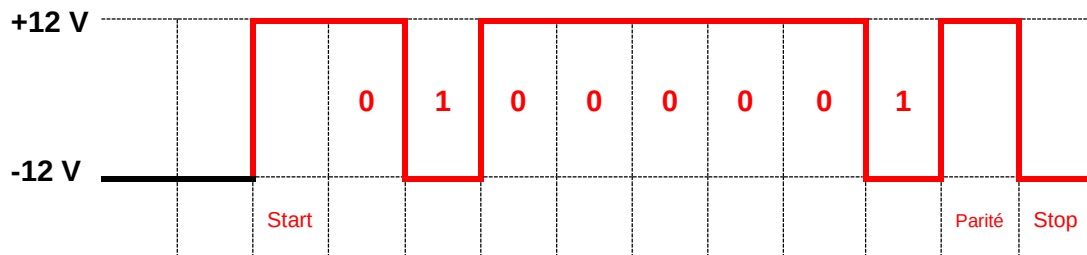
Quel est le type de logique utilisé (positive ou négative) ?

Question 4

Représentez ci-dessous la trame électrique d'une liaison série asynchrone permettant d'envoyer le caractère « é » (voir table ASCII) avec la configuration ci-dessous :

- Donnée sur 8 bits
- Parité paire
- 1 bit de stop

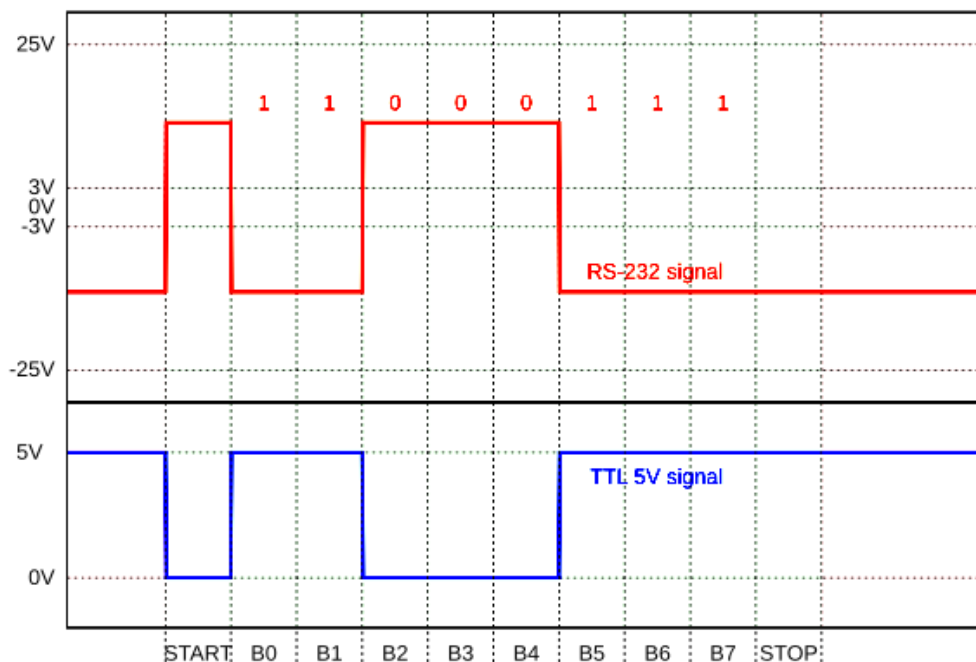
Selon la table ASCII : « é » vaut $130_{(10)}$ soit $\%1000\ 0010$.



La liaison série asynchrone est un moyen de communication entre deux systèmes électroniques facile à mettre en œuvre. Ainsi, la plupart des microcontrôleurs modernes disposent d'un circuit spécialisé (UART) leur permettant d'échanger des données avec l'extérieur de cette manière. Sur cette liaison (voir ci-dessous), une tension de 0 V correspond à un 0 logique. Une tension de +Vcc (+3,3 V ou +5 V selon la tension nominale du circuit) correspond à un 1 logique.

Cette liaison s'apparente à la liaison RS-232 dont certains PC sont encore équipés, mais elle s'en différencie par les tensions utilisées. À titre de rappel, en RS-232, un 1 logique correspond à une tension entre -3 V et -25 V et un 0 logique à une tension entre +3 V et +25 V.

Par conséquent, si vous souhaitez connecter une sortie série utilisant les niveaux TTL à votre ordinateur, il vous faudra un adaptateur spécialisé. Soit série TTL vers RS-232 ou mieux série TTL vers USB.



Question 5

D'après la trame électrique d'une liaison série asynchrone page précédente, retrouvez la donnée transmise en décimale.

Question 6

Quel mode de transmission nécessite la transmission du signal d'horloge ?

Question 7

Quel est le rôle du bit de parité dans une trame asynchrone ?

Question 8

*Soit une transmission d'une trame constituée de : 8 bits de données, d'un bit de start, d'un bit de stop et pas de parité.
Déterminez le nombre de bits de donnée transmis pendant une seconde, si le débit généré est de 9 600 bits/s.*

Déterminez le temps de transmission d'un fichier de 4 kio.

EXERCICE 2

Déterminer la valeur en décimal de la donnée transmise dans les figures suivantes

Longueur

☒ 7 bits

☐ 8 bits

Parité

☐ aucune

☐ paire

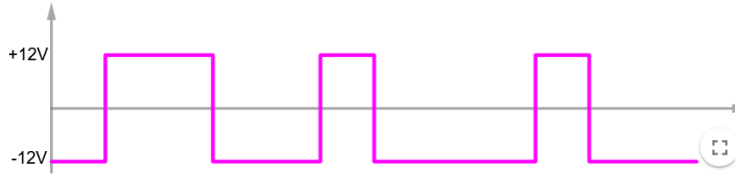
☒ impaire

Stop

☐ 1 bit

☒ 2 bits

☐ Logique ☒ Électrique



Longueur

☒ 7 bits

☐ 8 bits

Parité

☐ aucune

☒ paire

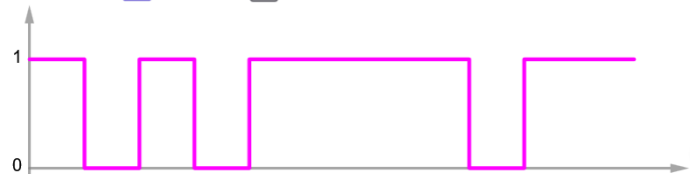
☐ impaire

Stop

☒ 1 bit

☐ 2 bits

☒ Logique ☐ Électrique



Longueur

☒ 7 bits

☐ 8 bits

Parité

☐ aucune

☐ paire

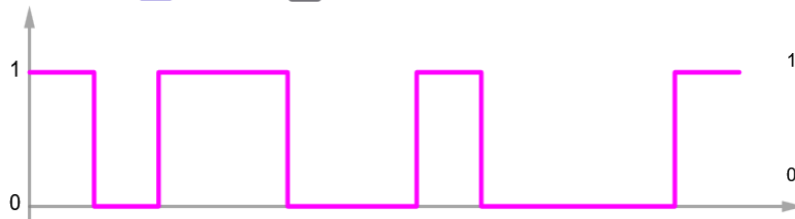
☒ impaire

Stop

☒ 1 bit

☐ 2 bits

☒ Logique ☐ Électrique



Longueur

☐ 7 bits

☒ 8 bits

Parité

☐ aucune

☒ paire

☐ impaire

Stop

☐ 1 bit

☒ 2 bits

☒ Logique ☐ Électrique



Longueur

☒ 7 bits

☐ 8 bits

Parité

☐ aucune

☐ paire

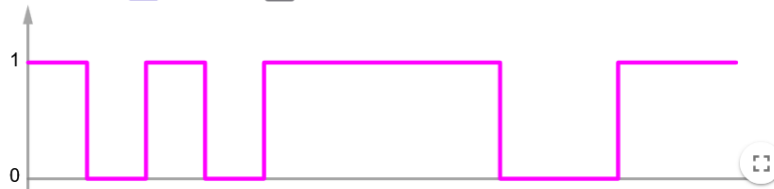
☒ impaire

Stop

☐ 1 bit

☒ 2 bits

☒ Logique ☐ Électrique



Longueur

☒ 7 bits

☐ 8 bits

Parité

☐ aucune

☐ paire

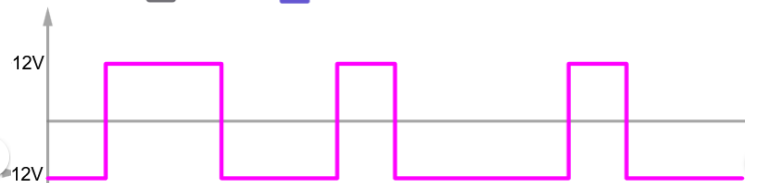
☒ impaire

Stop

☐ 1 bit

☒ 2 bits

☐ Logique ☒ Électrique



Longueur

☒ 7 bits
☐ 8 bits

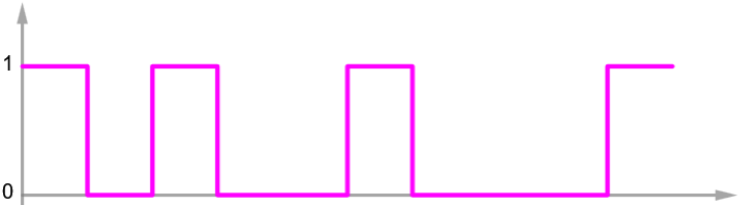
Parité

☒ aucune
☐ paire
☐ impaire

Stop

☒ 1 bit
☐ 2 bits

☒ Logique ☐ Électrique



Longueur

☒ 7 bits
☐ 8 bits

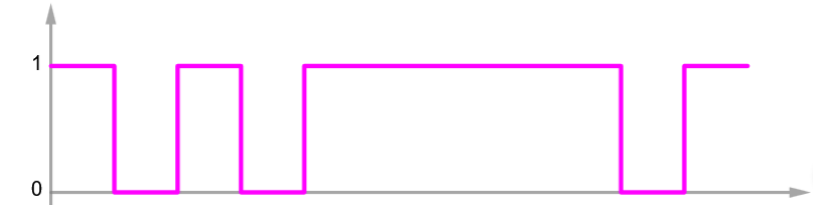
Parité

☐ aucune
☒ paire
☐ impaire

Stop

☒ 1 bit
☐ 2 bits

☒ Logique ☐ Électrique



Longueur

☒ 7 bits
☐ 8 bits

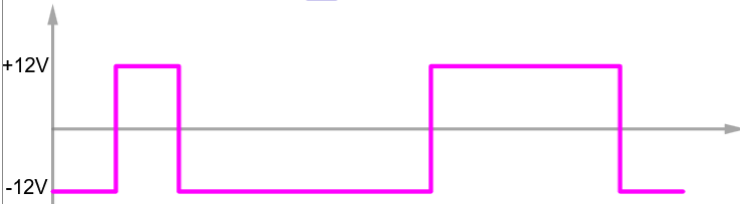
Parité

☒ aucune
☐ paire
☐ impaire

Stop

☒ 1 bit
☐ 2 bits

☐ Logique ☒ Électrique



Longueur

☒ 7 bits
☐ 8 bits

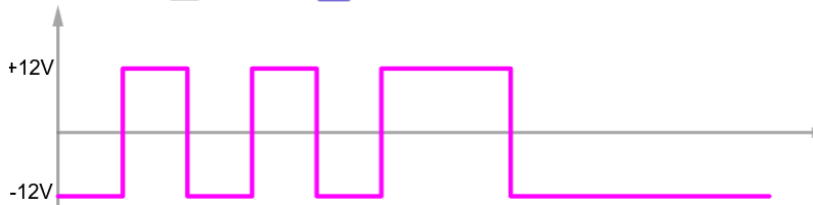
Parité

☐ aucune
☐ paire
☒ impaire

Stop

☒ 1 bit
☐ 2 bits

☐ Logique ☒ Électrique



Longueur

☒ 7 bits
☐ 8 bits

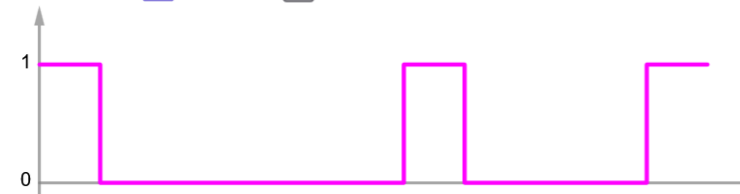
Parité

☐ aucune
☐ paire
☒ impaire

Stop

☒ 1 bit
☐ 2 bits

☒ Logique ☐ Électrique



Longueur

☐ 7 bits
☒ 8 bits

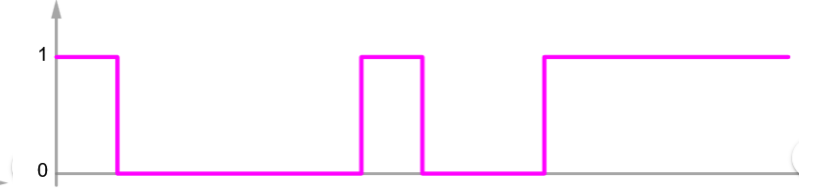
Parité

☐ aucune
☒ paire
☐ impaire

Stop

☒ 1 bit
☐ 2 bits

☒ Logique ☐ Électrique



Longueur

☒ 7 bits
☐ 8 bits

Parité

☐ aucune
☐ paire
☒ impaire

Stop

☒ 1 bit
☐ 2 bits

☒ Logique ☐ Électrique



Longueur

☒ 7 bits
☐ 8 bits

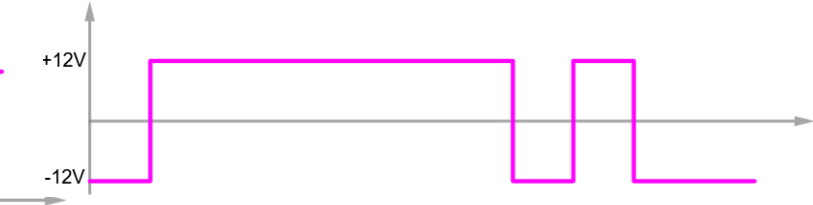
Parité

☒ aucune
☐ paire
☐ impaire

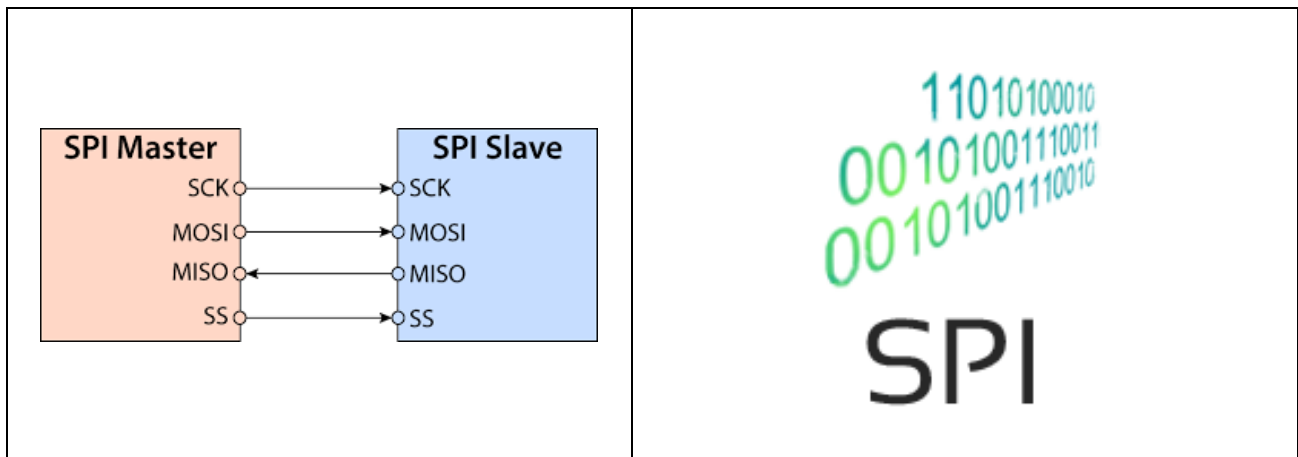
Stop

☐ 1 bit
☒ 2 bits

☐ Logique ☒ Électrique



Les protocoles de communication SPI



Objectifs du cours :

Ce cours traitera essentiellement les points suivants :

- Le bus SPI :
 - schéma de principe
 - chronogramme
- Compléments (comparaison I2C/SPI et bibliothèques SPI)

LE BUS SPI

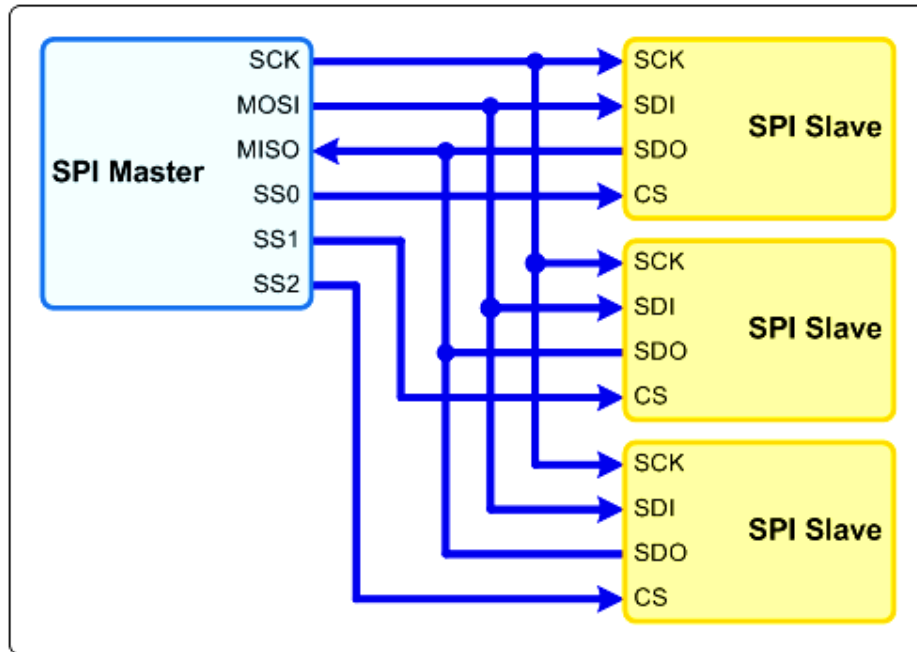
Dans les circuits d'interface le bus SPI (Serial Peripheral Interface) n'est pas seul et se trouve talonné depuis de nombreuses années par les circuits dotés d'une liaison série synchrone de type SPI. Cette appellation rencontre cependant des signaux et des chronogrammes qui sont beaucoup moins bien normalisés que ceux du bus I2C, tout simplement parce que la liaison du bus SPI ne fait l'objet d'aucune norme officielle.

Certains circuits indiquent qu'ils sont munis d'une interface SPI, mais vous pourrez aussi trouver comme appellation « Microwire » (qui est une marque déposée de National Semiconductor) ou bien encore « bus série trois fils ». Un bus de ce type permet la connexion, sous forme série maître-esclave, de plusieurs circuits disposant d'interfaces compatibles, avec seulement trois fils de liaisons.

Les dernières versions d'Arduino incluent une bibliothèque qui permet la communication avec les périphériques SPI.

SPI a des lignes labellisés « MOSI » : Master Output Slave Input, généré par le maître, des lignes labellisées « MISO » : Master Input Slave Output, généré par l'esclave et une ligne d'horloge (SCK : SPI Serial Clock). Ces trois lignes sont connectées à leurs lignes respectives sur un ou

plusieurs esclaves. Les esclaves sont identifiées grâce au signal de leur ligne SS (Slave Select).
Autres nommages possibles : SDO (Serial Data Out) et SDI (Serial Data Input).
La figure ci-dessous illustre les connexions SPI.



Maître SPI avec plusieurs esclaves SPI

Les numéros de broches à utiliser pour les broches SPI sont listés dans le tableau ci-dessous.

Broches numériques utilisées pour SPI		
Signal SPI	Carte Arduino Uno	Carte Arduino Mega
SS (choix de l'esclave)	10	S3
MOSI (entrées de données)	11	S1
MISO (sorties de données)	12	S0
SCLK (horloge)	13	S2

SCHÉMA DE PRINCIPE

La figure ci-dessous correspond au schéma de principe simplifié d'une liaison de données SPI, avec ses principaux composants.

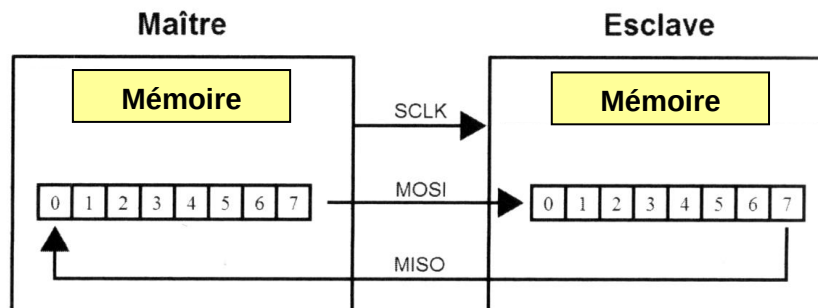


Schéma de principe simplifié d'une liaison SPI

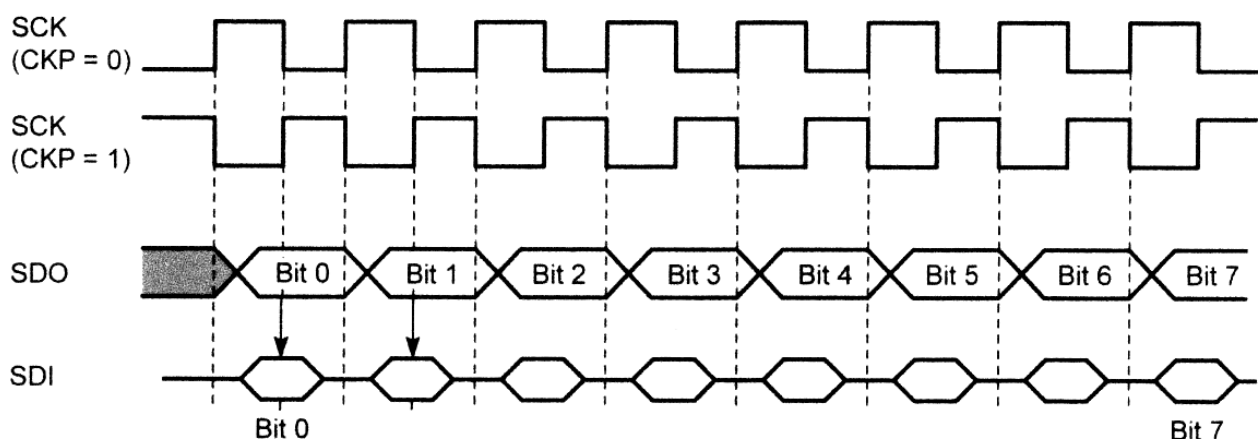
Deux registres à décalages sont généralement mis en œuvre. Ils peuvent être de type matériel ou logiciel, selon les dispositifs employés. Par exemple, le RasPi implémente son registre à décalage de façon logiciel alors que le MCP3008 (ADC) est équipé d'un registre à décalage matériel. Quelque soit leur mise en œuvre, ces deux registres à décalage forment un buffer circulaire interpuce, qui se trouve au cœur de la liaison SPI.

Les communications sont initiées par le maître, qui commence par sélectionner l'esclave. Au cours de chaque cycle d'horloge, le maître envoie un bit à l'esclave, qui le lit sur la ligne MOSI. En parallèle, l'esclave envoie un bit au maître, qui le lit sur la ligne MISO. Ces lectures et écritures simultanées entre le maître et l'esclave forment **une communication Full-Duplex**.

La fréquence de l'horloge dépend principalement des possibilités de réponse de l'esclave. Par exemple, le MCP3008 alimenté en 5 V est capable de gérer des fréquences de données allant jusqu'à 3,6 MHz.

CHRONOGRAMME

Toutes les liaisons SPI respectent le chronogramme général ci-dessous, mais quatre variantes existent au niveau de la prise en compte des données qui sont échangées.

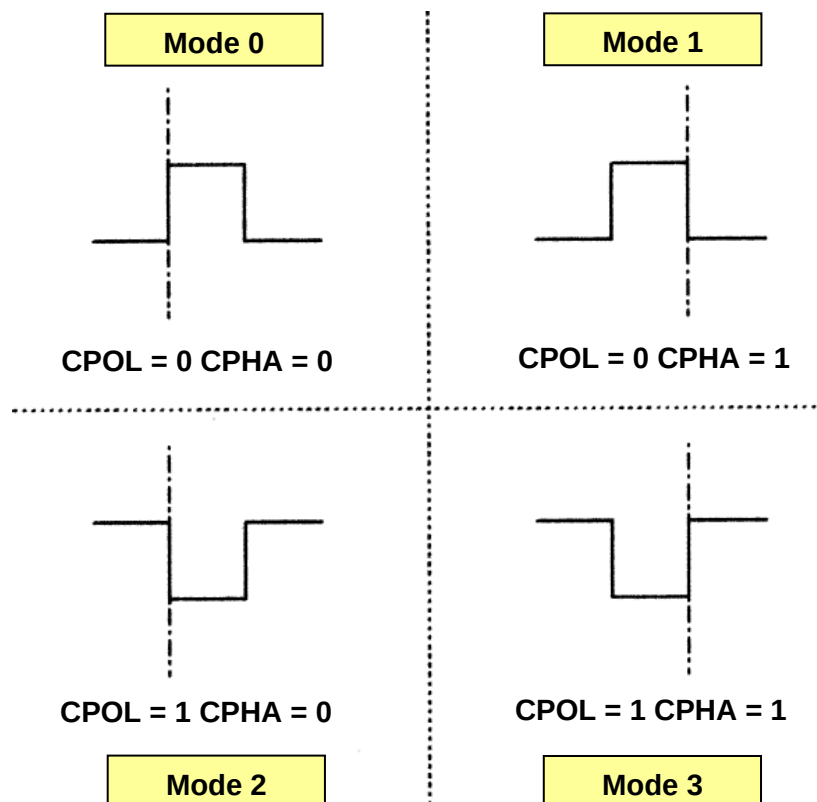


Chronogramme général d'un échange sur une liaison SPI

Ces variantes sont parfois appelées modes SPI et sont alors numérotées de 0 à 3. En présence d'un circuit à interface SPI, il vous suffira de comparer sa fiche technique avec le chronogramme pour savoir dans quel « mode » il fonctionne.

Ces quatre modes sont définis par la polarité de l'horloge, appelée CPOL (Clock POLarity), qui peut-être égale à 0 ou à 1, et par la phase de cette même horloge, appelée CPHA (Clock PHase), par rapport aux données, qui peut être aussi égale à 0 ou à 1. Cela conduit aux quatre possibilités (voir page suivante), qui indiquent sur quel front du signal d'horloge les données sont échantillonnées et peuvent donc être considérées comme valides.

Par défaut, le mode le plus courant est appelé mode 0, c'est-à-dire celui où CPOL et CPHA sont tous les deux nuls.



Les quatre modes de prise en compte des données de la liaison SPI

COMPLÉMENTS

La gestion d'une liaison SPI avec l'Arduino est facile à mettre en œuvre. Les micro-contrôleurs d'Atmel Atmega disposent d'une interface SPI dont les entrées/sorties se répartissent comme suit : SCK sur 13, MISO sur 12, MOSI sur 11 et /SS sur 10 pour les Uno.

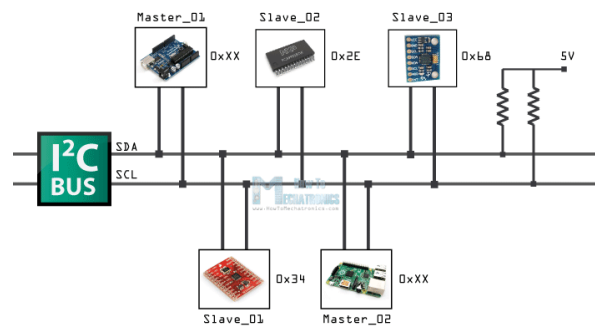
L'environnement de développement contient une bibliothèque nommée, SPI.h, qui permet une gestion logicielle très simple du protocole de dialogue des liaisons SPI.

Comparaison entre I2C et SPI : <http://www.maximintegrated.com/en/app-notes/index.mvp/id/4024>

Référence bibliothèque Arduino SPI : <http://arduino.cc/en/Reference/SPI>

Les protocoles de communication

I2C



Objectifs du cours :

Ce cours traitera essentiellement les points suivants :

- Le bus I2C :
 - protocole
 - en-tête
- Exemples avec le circuit Dallas DS1307 (RTC)
- La gestion des conflits
- Compléments (Wire Library pour Arduino, les adresses réservées, caractéristiques électriques)
- Exercices d'application

INTRODUCTION

Les normes I2C (Inter-Integrated Circuit) et SPI (Serial Peripheral Interface) ont été créées pour fournir un moyen simple de transférer des informations numériques entre des capteurs et des microcontrôleurs.

Les bibliothèques Arduino pour I2C et SPI facilitent l'utilisation de ces deux protocoles.

Le choix entre I2C et SPI est en général déterminé par les périphériques que l'on souhaite connecter. Certains appareils offrent les deux standards, mais habituellement un périphérique ou une puce ne supporte qu'une seule des deux normes.

I2C a l'avantage de n'avoir besoin que de deux connexions de signalisation (l'emploi de plusieurs périphériques sur les deux connexions est assez simple et on reçoit une confirmation que les signaux ont été correctement reçus).

L'inconvénient est que la vitesse des données est inférieure à celle de SPI et qu'elles ne peuvent voyager que dans un seul sens à la fois (Simplex), ce qui abaisse encore plus le débit si des communications bidirectionnelles sont nécessaires (Half Duplex). Il faut aussi connecter des résistances « pull-up » aux connexions pour s'assurer de la fiabilité de la transmission des signaux.

L'avantage de SPI est qu'il a un meilleur débit et qu'il a des connexions d'entrée et de sorties séparées, si bien qu'il peut envoyer et recevoir en même temps (Full Duplex). Il utilise une ligne supplémentaire par appareil pour sélectionner le périphérique actif et il faut donc plus de connexions si on a de nombreux appareils à connecter.

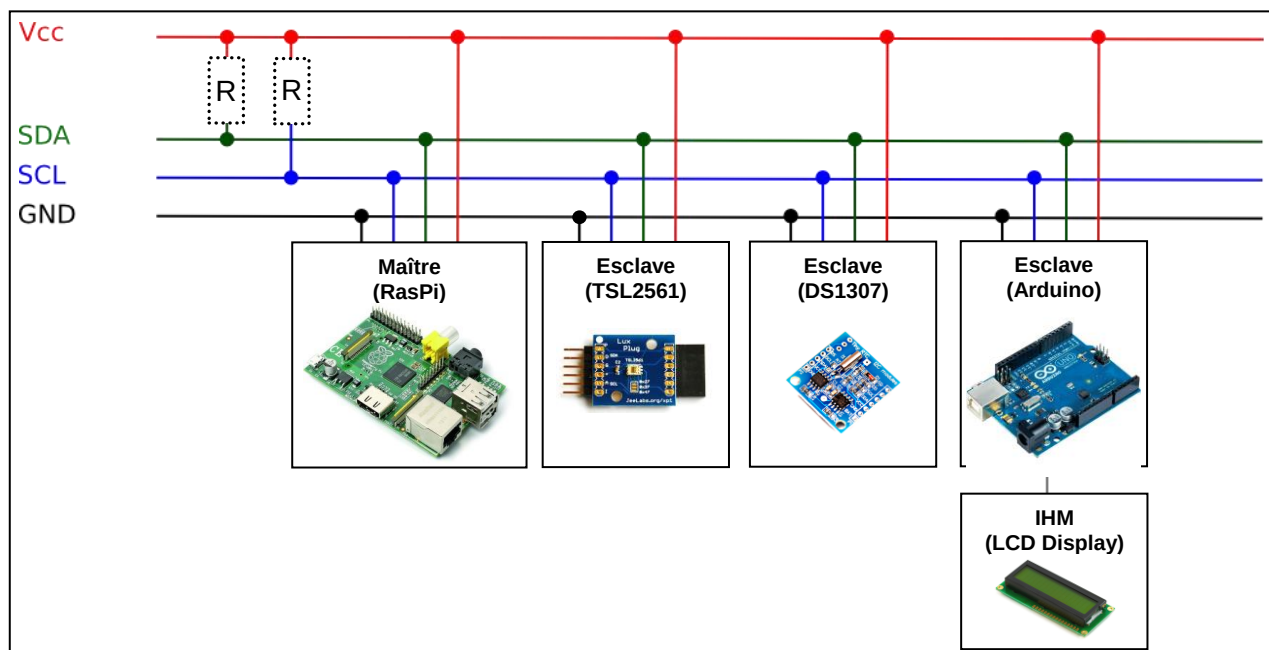
LE BUS I2C

Les deux connexions du bus I2C se nomment SCL (Serial Clock Line) et SDA (Serial Data line). Elles sont disponibles sur une carte standard Arduino (Uno) en employant la broche analogique 5 pour SCL qui fournit un signal d'horloge, et la broche analogique 4 pour SDA, qui s'occupe du transfert des données (sur la Mega, il faut utiliser la broche 20 pour SDA et la broche 21 pour SCL).

Pour le Raspberry Pi type B, il s'agit respectivement des broches GPIO2 et GPIO3 pour SDA et SCL repérées sur le « Pi Cobbler ».

Un périphérique sur le bus I2C est considéré comme le périphérique maître. Son travail consiste à coordonner le transfert des informations entre les autres périphériques (esclaves) qui sont connectés.

Il ne doit y avoir qu'un seul maître qui contrôle les autres composants auxquels il est connecté. La figure ci-dessous montre un maître I2C avec plusieurs esclaves I2C.



Maître I2C avec plusieurs esclaves I2C

Les périphériques I2C ont besoin d'une masse commune pour communiquer.

Les périphériques esclaves sont identifiés par leur numéro d'adresse. Chaque esclave doit avoir une adresse unique. Certains appareils I2C ont une adresse fixe alors que d'autres permettent que l'on configure leur adresse en définissant les broches à HIGH ou LOW ou en envoyant des commandes d'initialisation (la solution est matérielle ou logicielle).

Remarques :

L'Arduino utilise des valeurs sur 7 bits pour spécifier les adresses I2C. Certaines notices techniques de périphériques emploient des adresses sur 8 bits, dans ce cas il faut diviser par deux pour obtenir la valeur correcte sur 7 bits.

La bibliothèque Arduino Wire masque toutes les fonctionnalités de bas niveau du I2C et permet l'emploi de commandes simples pour initialiser les appareils et communiquer avec eux.

Sur la figure (page précédente) les pull-up sont représentées en pointillées, elles ne sont pas nécessaires pour l'Arduino et le Raspberry car les résistances de tirage sont intégrées sur les cartes.

Le bus I2C permet d'échanger des informations sous forme série avec un débit pouvant atteindre 100 kbps (standard mode), 400 kbps (fast mode) ou 3,2 Mbps (high speed mode) pour les versions les plus récentes.

Ses points forts sont les suivants :

- c'est un bus série bifilaire (8 bits) utilisant une ligne de donnée SDA et une ligne d'horloge SCL ;
- le bus est multi maîtres ;
- chaque abonné dispose d'une adresse codée sur 7 bits, on peut donc connecter simultanément 128 abonnés d'adresses différentes sur le même bus (sous réserve de ne pas le surcharger électriquement = la charge) ;
- un acquittement est généré pour chaque octet de donnée transféré ;
- un procédé permet de ralentir l'équipement le plus rapide pour s'adapter à la vitesse de l'équipement le plus lent lors d'un transfert ;
- le nombre maximal d'abonné n'est limité que par la charge capacitive maximale du bus qui peut être de 400 pF ;
- les niveaux électriques permettent l'utilisation des circuits en technologies CMOS, NMOS ou TTL.

PROTOCOLE

La figure ci-dessous montre le principe adopté au niveau des étages d'entrées/sorties des circuits d'interfaces du bus I2C.

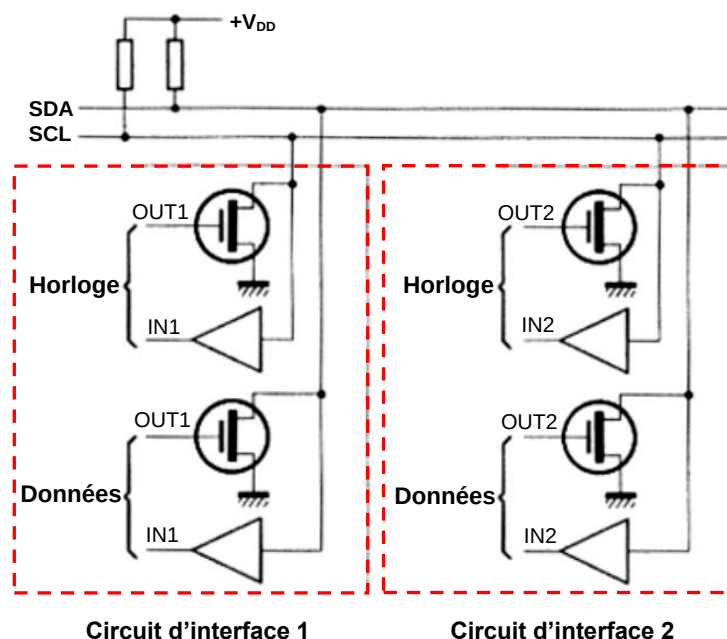


Schéma de l'interface matérielle des circuits compatibles du bus I2C

La partie entrée n'appelle aucune remarque particulière, en revanche la partie sortie fait appel à une configuration à drain ouvert (l'équivalent en MOS du classique collecteur ouvert) et qui permet de réaliser des ET câblés par simple connexion sur la ligne SDA ou SCL, des sorties de tous les circuits.

Aucune charge n'étant prévue dans ces derniers, une résistance de rappel (pull-up) à une tension positive doit être mise en place.

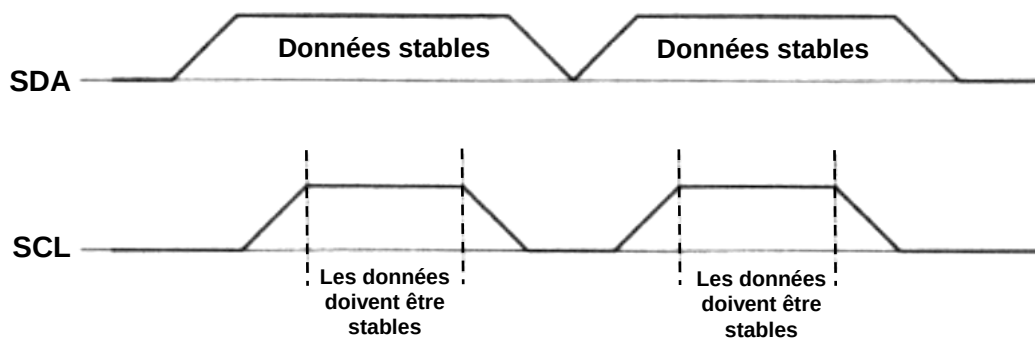
Étant entendu que nous travaillons en logique positive, c'est-à-dire qu'un niveau haut correspond à une tension plus élevée qu'un niveau bas.

Lorsqu'aucun abonné n'émet sur le bus, les lignes SDA et SCL sont au niveau haut qui est leur état de repos.

Quand une ligne (SDA ou SCL) est au repos (niveau 1), on peut la forcer à 0.

Quand une ligne (SDA ou SCL) est au niveau 0, on ne peut pas la forcer à 1.

CHRONOGRAMME FONDAMENTAL



Chronogramme fondamental du bus I2C

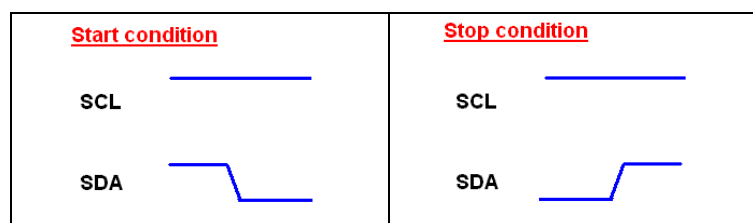
Le chronogramme ci-dessus résume le principe fondamental d'un transfert de données :

Une donnée n'est considérée comme valide sur le bus que lorsque le signal SCL est à l'état haut.

L'émetteur doit donc positionner la donnée à émettre lorsque SCL est à l'état bas et la maintenir tant que SCL est à l'état haut.

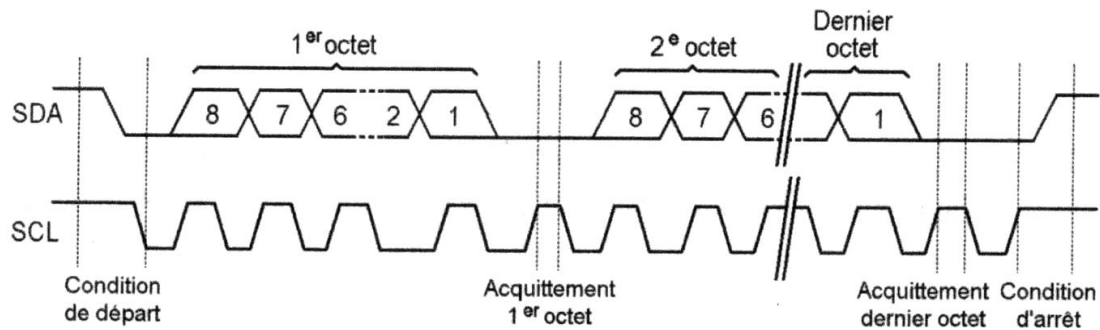
Comme la transmission s'effectue sous forme série, une information de début et de fin doit être prévue. L'information de début se nomme **START** et l'information de fin **STOP**.

Une condition de départ est réalisée lorsque la ligne SDA passe du niveau haut au niveau bas alors que SCL est au niveau haut. Réciproquement, une condition d'arrêt est réalisée lorsque SDA passe du niveau bas au niveau haut alors que SCL est au niveau haut.



Les données sont envoyées par paquets de huit bits (ou octets). Le bit de poids fort est envoyé le premier, chaque octet est suivi par un bit d'acquittement (ACK) de la part du destinataire.

Le processus fonctionne de la façon ci-dessous.



Chronogramme d'un échange sur bus I2C

La ligne SCL est pilotée par l'initiateur de l'échange ou maître.

Le chronogramme ci-dessus montre d'abord une condition de départ, généré par le maître du bus à cet instant. Elle est suivie par le premier octet de données, poids fort en tête. Après le huitième bit, l'émetteur qui est aussi le maître dans ce cas met sa ligne SDA au niveau haut, c'est-à-dire au repos mais continue à générer l'horloge sur SCL.

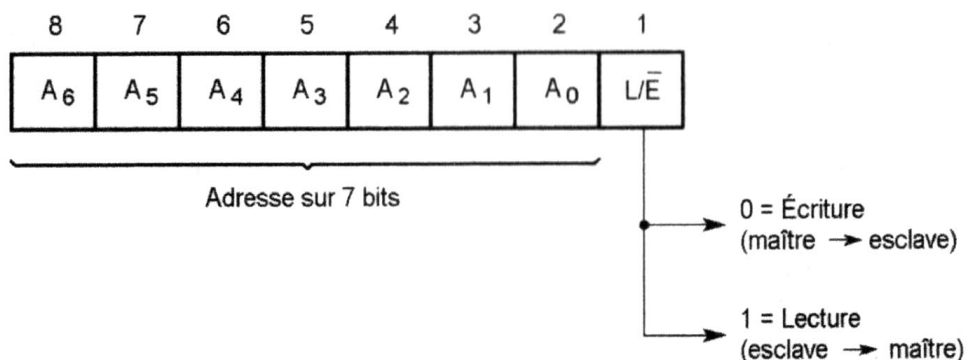
Pour acquiescer l'octet, le récepteur doit alors forcer la ligne SDA au niveau bas pendant l'état haut de SCL qui correspond à cet acquiescement, prenant en quelque sorte la place d'un neuvième bit.

Le processus peut alors continuer avec l'octet suivant et se répéter autant de fois que nécessaire pour réaliser un échange d'informations complexes.

Lorsque cet échange est terminé, le maître génère une condition d'arrêt.

EN-TÊTE

La figure ci-dessous montre le contenu du premier octet qui est toujours présent en début d'échange. Ses sept bits de poids forts contiennent l'adresse du destinataire, ce qui autorise 128 combinaisons différentes. Le bit de poids faible indique si le maître va réaliser une lecture ou une écriture. Si ce bit est à zéro, le maître va écrire dans l'esclave ou lui envoyer des données. Si ce bit est à un, le maître va recevoir des données de l'esclave.



Contenu de l'octet d'en-tête de l'échange sur le bus I2C

Lorsqu'un maître désire effectuer plusieurs échanges à destination d'esclaves d'adresses différentes, il n'est pas obligé de terminer le premier échange par une condition d'arrêt mais peut les enchaîner en générant une condition de départ dès la fin d'un échange.

EXEMPLES AVEC LE CIRCUIT DALLAS DS1307 (RTC)

Le circuit Dallas DS1307 est une horloge temps réel (Real Time Clock), qui fournit secondes, minutes, heures, jours, dates, mois et années.

Les années bissextiles sont prises en compte (jusqu'en 2100).

Le DS1307 travaille avec un débit binaire dans le mode standard.

L'adresse I2C (7 bits) du DS1307 est : 1101000 (adresse fixée par le constructeur et non modifiable).

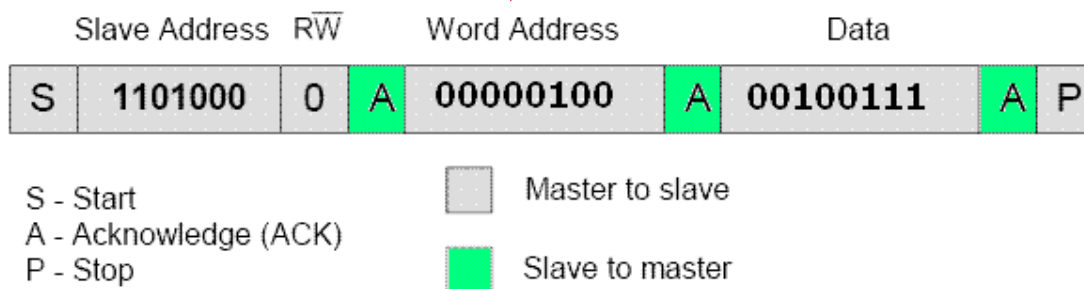
EXEMPLE D'ÉCRITURE

L'émetteur est le maître et le récepteur est l'esclave.

Le registre d'adresse 04h du DS1307 contient la date (voir datasheet du DS1307).

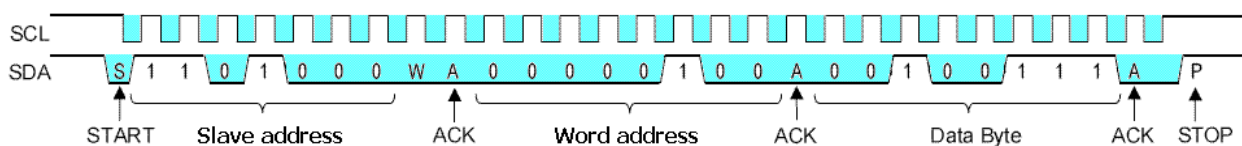
Pour régler le calendrier au 27 du mois, il faut écrire la donnée 27 (en code BCD) dans le registre d'adresse 04h du DS1307.

Le bus I2C utilise le protocole suivant :



- 1) Pour initier le dialogue, le maître crée une condition Start.
- 2) Le maître envoie l'adresse de l'esclave (1101000) suivi du bit 0 (bit Write).
- 3) L'esclave répond (accusé de réception : bit ACKnowledge).
- 4) Le maître envoie l'adresse du registre (04h) à écrire.
- 5) L'esclave répond (accusé de réception : bit ACKnowledge).
- 6) Le maître envoie la donnée (27) à écrire.
- 7) L'esclave écrit la donnée puis envoie un accusé de réception (bit ACKnowledge).
- 8) Le maître termine le dialogue avec une condition Stop.

Le bus I2C est maintenant libre (SCL = 1, SDA = 1 = niveaux de repos).



Chronogramme complet de l'échange sur bus I2C avec l'esclave DS1307

Question

D'après l'exemple ci-dessus, combien de temps faut-il pour le transfert en écriture ?

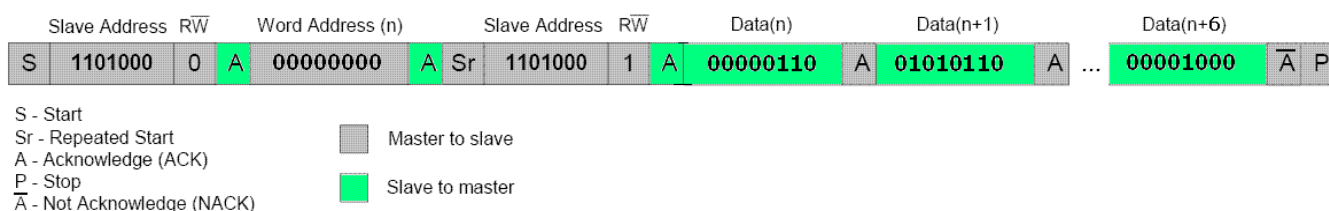
.....

EXEMPLE DE LECTURE

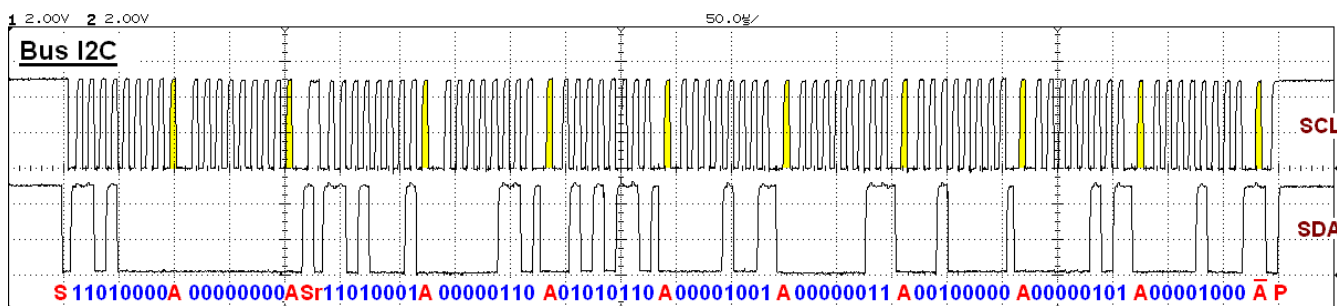
L'émetteur est l'esclave et le récepteur est le maître.

Les registres d'adresses 00h à 06h du DS1307 contiennent respectivement les secondes, minutes, heures, jours, dates, mois et années (voir datasheet du DS1307).

Voici comment lire, d'une seule traite, le contenu des registres d'adresses 00h à 06h du DS1307 :



Chronogramme correspondant :



Chronogramme complet de l'échange sur bus I2C avec l'esclave DS1307

- 1) Pour initier le dialogue, le maître crée une condition Start.
 - 2) Le maître envoie l'adresse de l'esclave (1101000) suivi du bit 1 (bit Write).
 - 3) L'esclave répond (accusé de réception : bit ACKnowledge).
 - 4) Le maître envoie l'adresse du registre (0x00) à lire.
 - 5) L'esclave répond (accusé de réception : bit ACKnowledge).
 - 6) Le maître émet une condition Repeated Start.
 - 7) Le maître envoie l'adresse de l'esclave (1101000) suivi du bit 1 (bit Read).
 - 8) L'esclave répond (accusé de réception : bit ACKnowledge).
 - 9) L'esclave envoie le contenu du registre d'adresse 0x00 au maître.
 - 10) Le maître répond (accusé de réception : bit ACKnowledge).
 - 11) L'esclave envoie le contenu du registre d'adresse 0x01 (automatiquement incrémenté) au maître.
 - 12) Le maître répond (accusé de réception : bit ACKnowledge).
 - 13) L'esclave envoie le contenu du registre d'adresse 0x02 (automatiquement incrémenté) au maître.
 - 14) Le maître répond (accusé de réception : bit ACKnowledge).
 -
 -
 - 21) L'esclave envoie le contenu du registre d'adresse 0x06 (automatiquement incrémenté) au maître.
 - 22) Le maître répond (accusé de réception : bit Not ACKnowledge).
 - 23) Le maître termine le dialogue avec une condition Stop.
- Le bus I2C est maintenant libre (SCL = 1, SDA = 1 = niveaux de repos).

LA GESTION DES CONFLITS

La structure du bus I2C a été conçue pour pouvoir y accueillir plusieurs maîtres. Se pose alors le problème commun à tous les réseaux utilisant un canal de communication unique : la prise de parole. En effet, chaque maître pouvant prendre possession du bus dès que celui-ci est libre, il existe la possibilité de que deux maîtres prennent la parole en même temps. Si cela ne pose pas de problème sur le plan électrique grâce l'utilisation de collecteurs ouverts, il faut pouvoir détecter cet état de fait pour éviter un conflit logique et donc éviter une collision entraînant la corruption des données transmises.

PRINCIPE

Pour prendre le contrôle du bus, un maître potentiel doit d'abord vérifier que celui-ci soit libre, et qu'une condition d'arrêt ait bien été envoyée depuis au moins $4,7\mu s$. Mais il reste la possibilité que plusieurs maîtres prennent le contrôle du bus simultanément.

Chaque circuit vérifie en permanence l'état des lignes SDA et SCL, y compris lorsqu'ils sont eux même en train d'envoyer des données. On distingue alors plusieurs cas :

① Les différents maîtres envoient les mêmes données au même moment :

Les données ne sont pas corrompues, la transmission s'effectue normalement, comme si un seul maître avait parlé. Ce cas est rare.

② Un maître impose un '0' sur le bus :

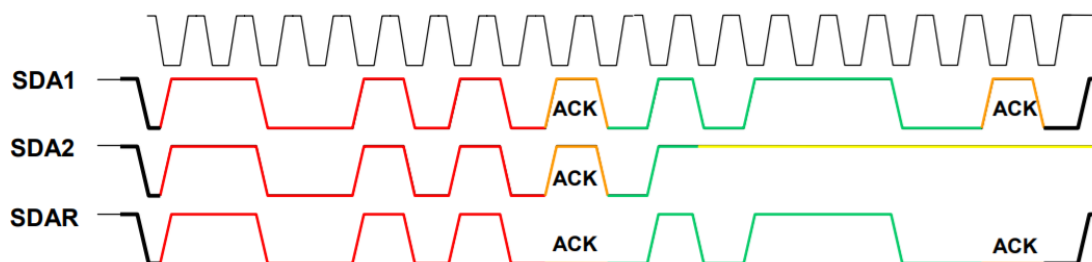
Il relira forcément '0' et continuera à transmettre. Il ne peut pas alors détecter un éventuel conflit.

③ Un maître cherche à appliquer un '1' sur le bus :

Si il ne relit pas un niveau '1', c'est qu'un autre maître a pris la parole en même temps. Le premier perd alors immédiatement le contrôle du bus, pour ne pas perturber la transmission du second. Il continue néanmoins à lire les données au cas où celles-ci lui auraient été destinées.

Exemple :

Soit le chronogramme ci-dessous.



SDA1 : Niveaux de SDA imposés par le maître n°1

SDA2 : Niveaux de SDA imposés par le maître n°2

SDAR : Niveaux de SDA réels résultants lus par les deux maîtres

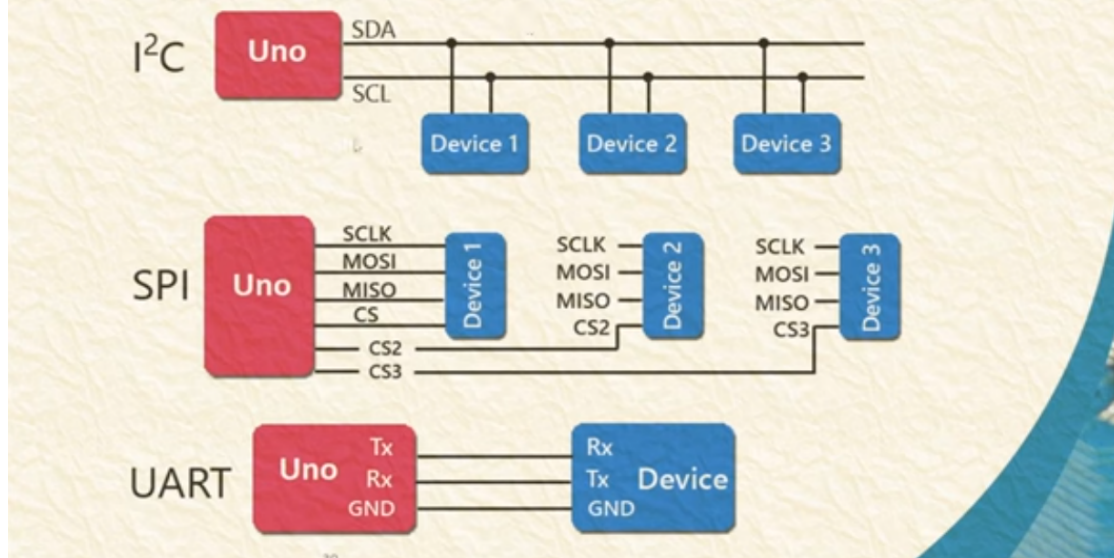
Le premier octet est transmis normalement car les deux maîtres imposent les mêmes données. Le bit ACK est mis à '0' par l'esclave.

Lors du deuxième octet, le maître n°2 cherche à imposer un '1' (SDA2) , mais relit un '0' (SDAR), il perd alors le contrôle du bus et devient esclave. Il reprendra le contrôle du bus, lorsque celui-ci sera de nouveau libre.

Le maître n°1 ne voit pas le conflit et continue à transmettre normalement.

Au total, l'esclave a reçu les données du maître n°1 sans erreurs et le conflit est passé inaperçu.

Comparison



Comparison

Parameters	UART	I2C	SPI
Protocol	Asynchronous	Synchronous	Synchronous
Wires Used	2 Wires	2 Wires	4+ wires
Maximum of Masters	1	Unlimited	1
Maximum of Slaves	1	127	4
Maximum of distance	15 m @20Kbps	1 meter (in board)	20 cm (in board)
Maximum speed	300, 600, 1200, 2400, 4800, 9600, 19200 38400, 57600, 115200, 230400 460800, and 921600 bit per sec (baud)	Standard mode = 100 Kbps Fast mode = 400 Kbps High mode = 3.4 Mbps Ultra mode = 5 Mbps	5 Mbps

EXERCICES D'APPLICATION

EXERCICES N°1

Question 1

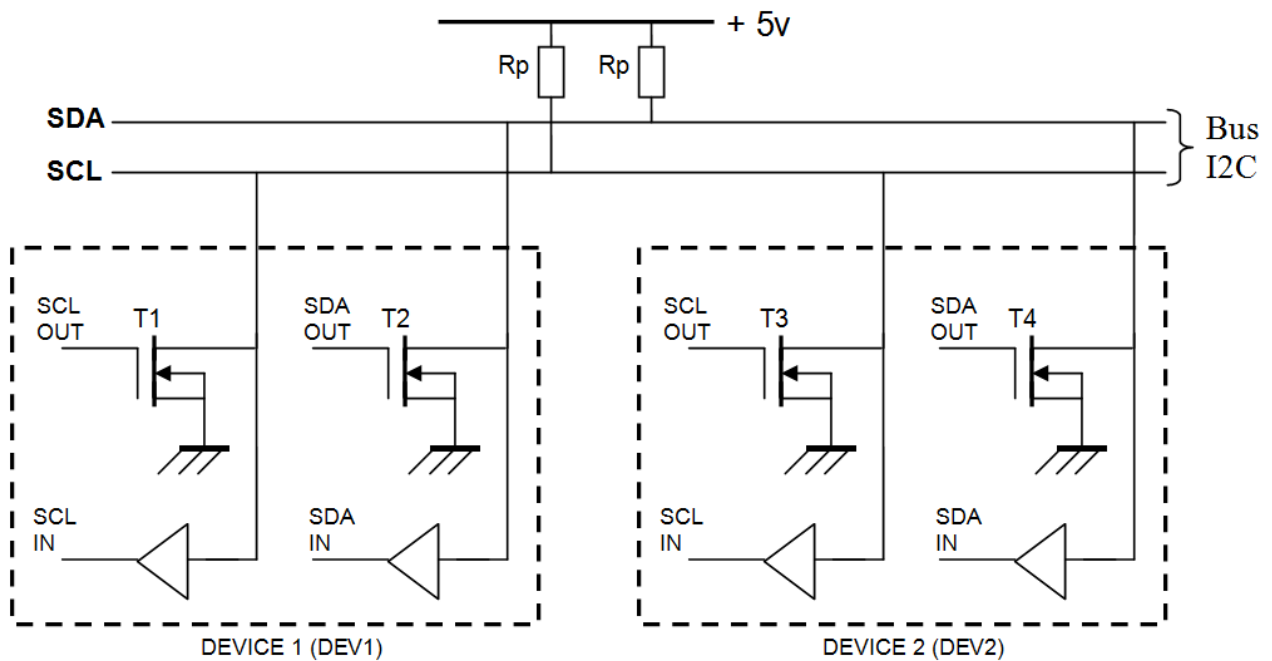
D'après le circuit page suivante, DEV1 désire prendre la parole (START). Avant de pouvoir émettre, **il doit s'assurer que le bus est libre, en effectuant un contrôle de l'état logique des signaux SDA et SCL**. Donnez l'équation logique de EMISSION = $f(\text{SCL IN}, \text{SDA IN})$ autorisant l'émission.

.....

Question 2

Lors d'une prise de parole du circuit DEV1 (START), indiquez, dans l'ordre chronologique, l'état des transistors T1 et T2.

.....



Question 3

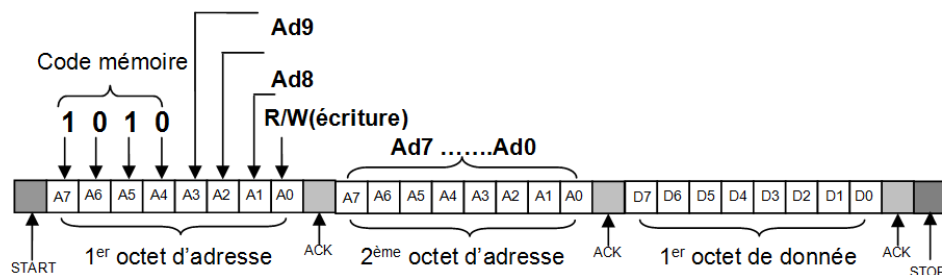
Dans quel état doit se trouver T2 afin de signifier au destinataire de la donnée, que le dernier bit a été transmis ?

Question 4

Le circuit DEV1 désire clôturer la prise de parole (STOP). Indiquez, dans l'ordre chronologique, l'état des transistors T1 et T2.

Question 5

Dans le cas d'un circuit nouvelle génération haut débit (3,2 Mbps), quel temps mettra un circuit pour transmettre un octet (hors temps de START et de STOP) selon le schéma ci-dessous ?



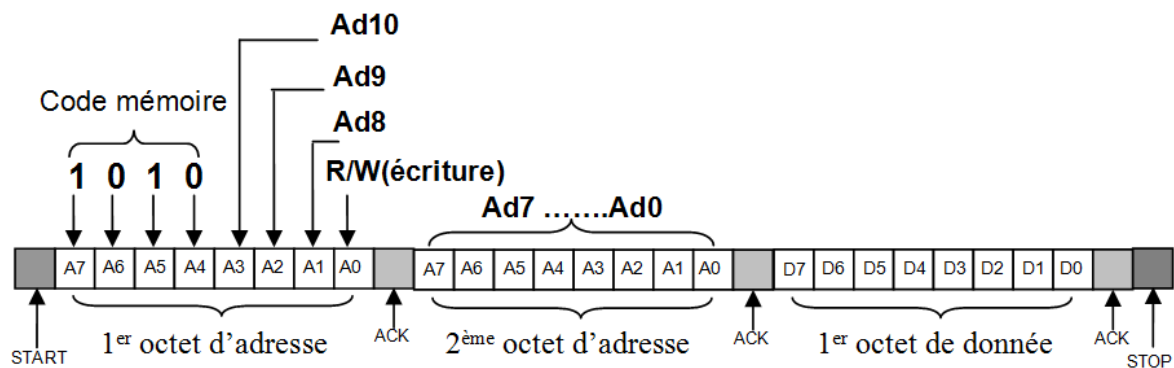
EXERCICES N°2

Lorsque l'adressage de la mémoire nécessite plus de 7 bits d'adresse, on distingue les 8 bits de la partie basse de l'adresse (Ad7 à Ad0) et la partie haute comprenant 3 bits (Ad8 à Ad10).

Le protocole I2C fait appel à un code spécifique (1010) lorsqu'il s'agit de communiquer avec une mémoire compatible I2C.

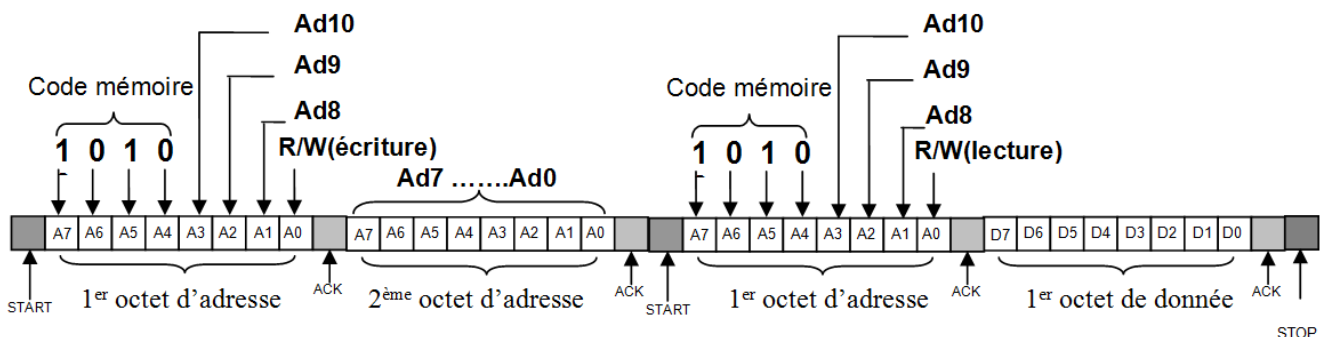
Principe d'écriture dans une mémoire avec adressage effectué sur 2 octets :

Le maître pointe l'adresse de la mémoire (2 octets) en mode écriture, puis écrit la donnée (1 octet).



Principe de lecture dans une mémoire avec adressage effectué sur 2 octets :

Le maître commence par pointer l'adresse de la mémoire (2 octets) en **mode écriture**, puis pointe la partie haute de l'adresse (1 octet) en mode lecture, avant de lire la donnée (1 octet). Lors de la réception de la donnée, c'est le maître qui devra accuser réception de la donnée en envoyant à la mémoire, un bit ACK.



Question 1

On désire écrire les données suivantes : \$A7 et \$5C dans les mémoires respectivement d'adresse \$01F et \$5D1.

Établissez les trames (en binaire) des informations que le maître devra envoyer sur la ligne **SDA** du bus I2C. Répondez en complétant les trames ci-dessous.

Écriture : adresse \$01F, donnée \$A7

[illegible]

Écriture : adresse \$5D1, donnée \$5C

[illegible]

Question 2

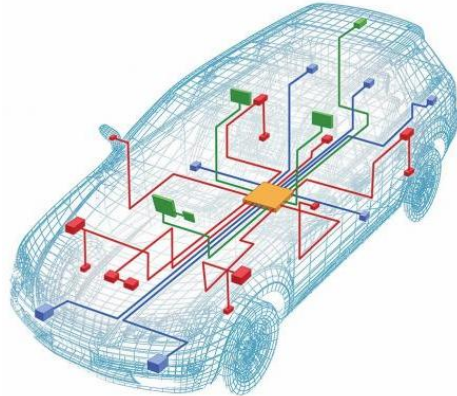
On désire lire la donnée \$8D stockée à l'adresse \$2FA.

Établissez la trame (en binaire) des informations que le maître devra envoyer sur la ligne **SDA** du bus I2C. Répondez en complétant la trame ci-dessous.

Lecture : donnée \$8D stockée à l'adresse \$2FA

[illegible][illegible]

Les protocoles de communication CAN-Bus



Objectifs du cours :

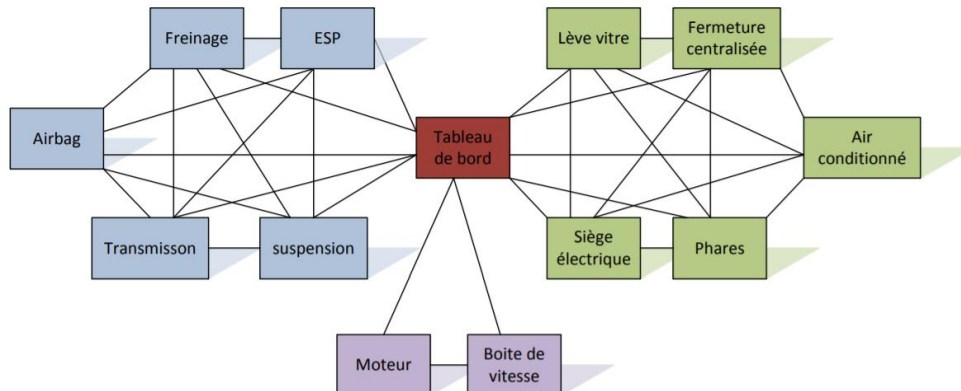
Ce cours traitera essentiellement les points suivants :

- Présentation du bus CAN
 - Le bus CAN et le modèle OSI
 - Propriétés physiques du bus CAN
 - le bus CAN embarqué dans les véhicules
 - niveau dominant et niveau récessif
 - arbitrage
 - version du bus CAN et niveaux électriques
 - les trames du bus CAN
 - constitution d'une trame de données
 - fiabilité et protection du bus CAN
 - Exercices
-

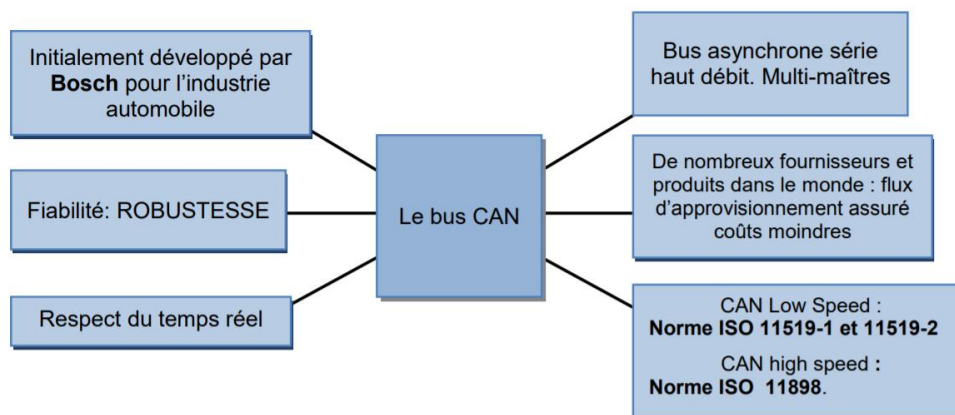
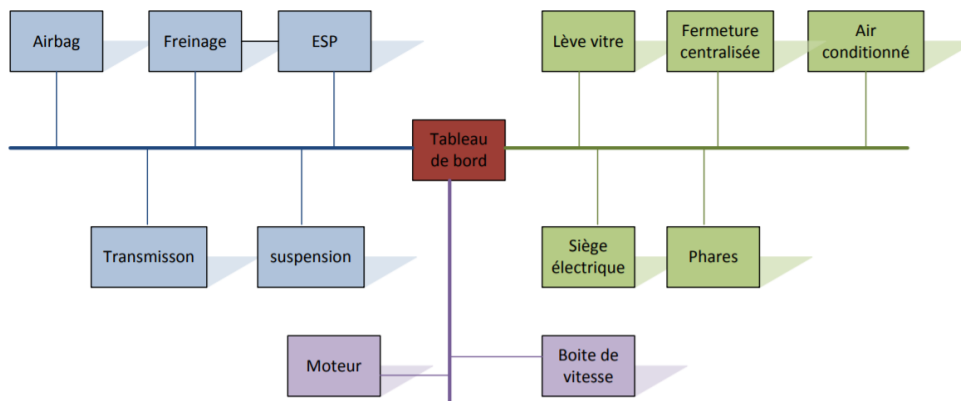
PRESENTATION

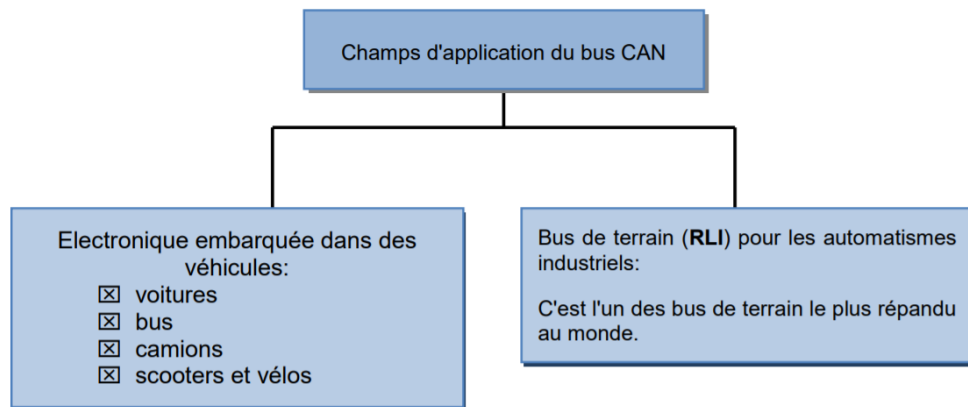
La sécurité et le confort des utilisateurs dans les véhicules ont entraîné une augmentation des organes électroniques.

Le câblage point à point (organe à organe) est devenu très complexe et très couteux en câble. On est passé de 200 m à 2000 m de câble dans un véhicule entre 1960 et 2000.



D'où la nécessité d'utiliser le multiplexage qui permet la communication à tour de rôle sur le même câble.

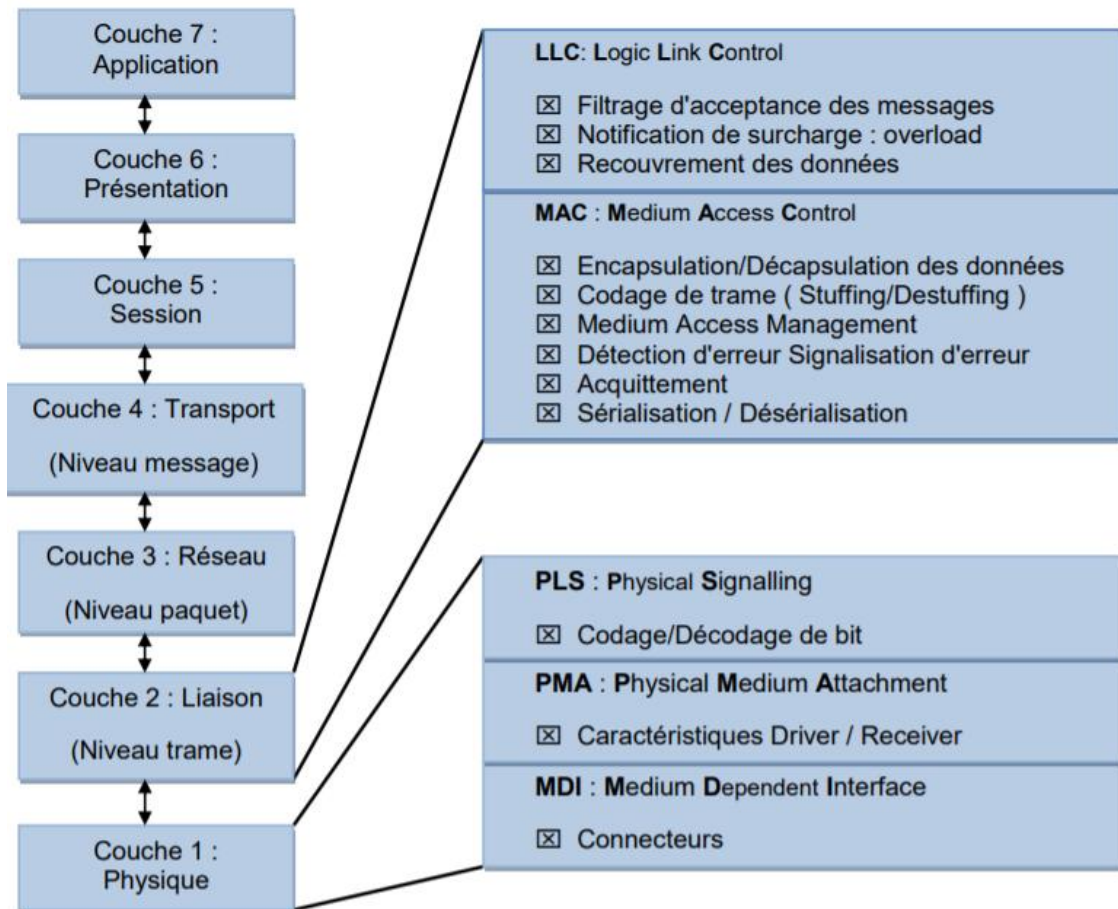




LE BUS CAN ET LE MODELE OSI

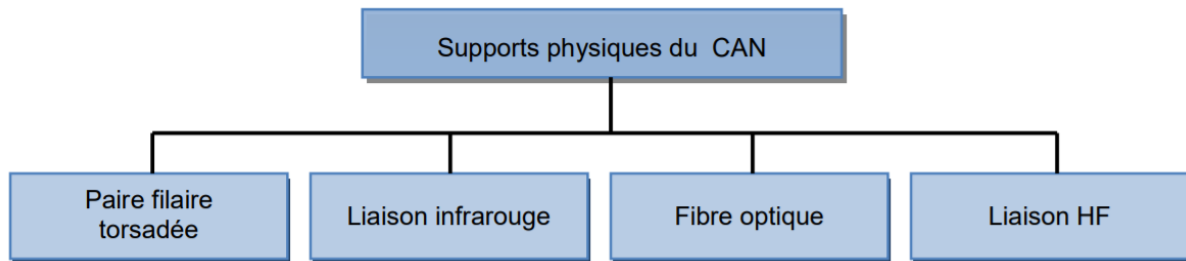


Le protocole CAN (Control Area Network) ne définit que la couche liaison de données (couche 2) et la couche physique (couche 1).



La couche 7 (APPLICATION) peut être implémentée sous forme d'un protocole supplémentaire (CANOpen par exemple).

PROPRIETES PHYSIQUES DU BUS CAN

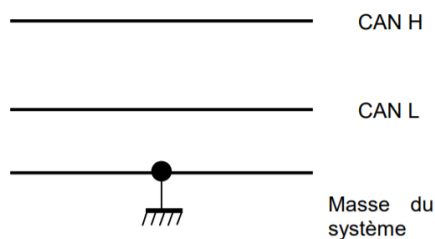


LE BUS CAN EMBARQUE DANS LES VEHICULES

La transmission des données est effectuée sur une paire filaire différentielle.



La ligne est constituée de 2 fils de données ainsi que de la masse.



Le **CAN** est un bus de terrain, soumis à des parasites importants. La transmission en paire différentielle permet de s'affranchir de ces problèmes car les montages différentiels ont un fort taux de réjection en mode commun CMRR.

Les débits varient en fonction de la longueur d'une paire torsadée :

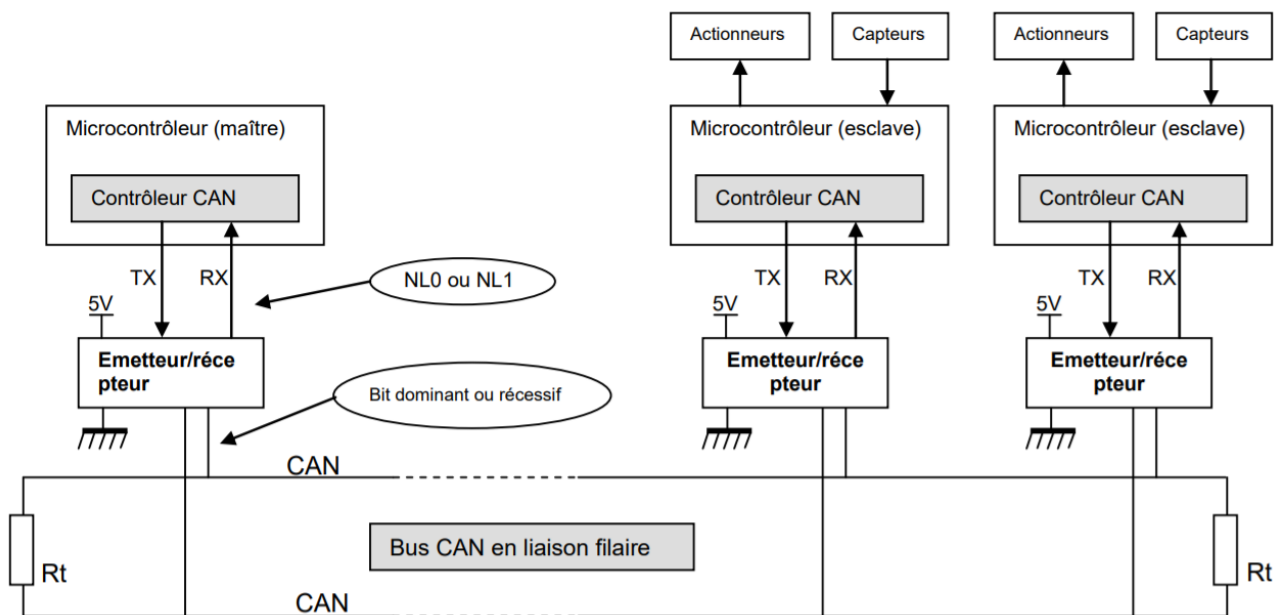
Débit	Longueur
1 Mbits/s	40m
500 Kbits/s	100m
100 Kbits/s	500m
20 Kbits/s	1000m

En pratique, il y a trois bus CAN différents dans un véhicule, à des débits différents :

Un bus rapide pour gérer la sécurité (freinage, ABS, détection chocs, airbags...) ;

Un bus à vitesse moyenne pour gérer le moteur (commandes et capteurs) ;

Un bus lent pour gérer tous les accessoires (lampes, moteurs d'asservissements, boutons...).



Synoptique d'une liaison filaire CAN intégrée dans un véhicule

Rôle des résistances de terminaison :

Si l'on fait circuler des signaux de tension sur le bus, sans résistance de terminaison (R_t) de ligne, les signaux vont se réfléchir sur les extrémités et vont créer des parasites qui risquent de perturber les émissions suivantes sur le bus (identique à une onde qui rebondirait contre un mur).

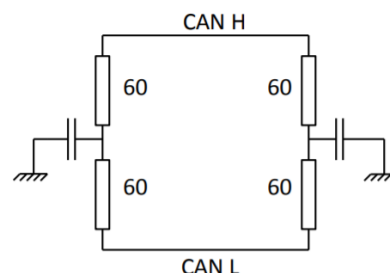
Pour éviter ces phénomènes de signal réfléchi en bout de câble, on place à l'extrémité une impédance identique à celle du câble. On trouvera donc à chaque extrémité du réseau deux résistances de 120 Ω . Ces résistances de fin de ligne sont intégrées aux extrémités du réseau CAN dans deux participants, en fonction de la topologie et l'architecture du réseau.

Un contrôle rapide de la continuité du réseau peut-être fait en mesurant la résistance entre CAN – H et CAN – L (hors tension et tous les calculateurs branchés).

On mesure deux résistances de 120 Ω en parallèles, 60 Ω .

La mesure de toute autre valeur est une anomalie :

- ☒ $R > 60 \Omega \rightarrow$ coupure de ligne ou si $R = 120\Omega$ absence d'une des deux résistances de terminaison.
- ☒ $R < 60 \Omega \rightarrow$ ligne en court-circuit, ou plus de deux résistances de terminaison dans le réseau.



NIVEAU DOMINANT ET NIVEAU RECESSIF

Un niveau électrique **récessif** sur le bus CAN correspond à un **niveau logique haut** sur Tx.

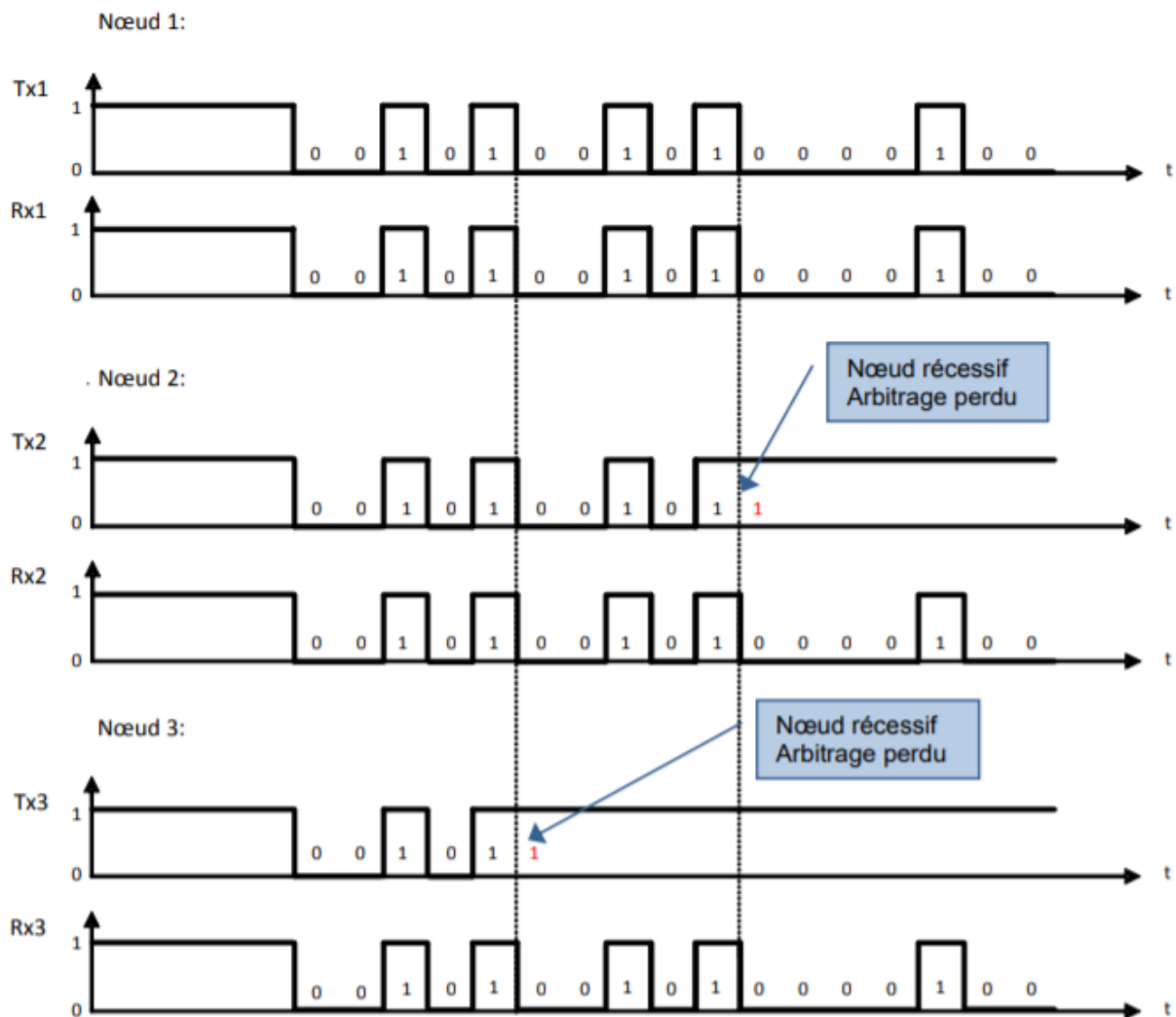
Un niveau électrique **dominant** sur le bus CAN correspond à un **niveau logique bas** sur Tx.

ARBITRAGE

Tout conflit de bus est résolu par le mécanisme du "**ET câblé**", c'est-à-dire qu'un état dominant écrase un état récessif. Si plusieurs nœuds débutent leur trame en même temps, le premier qui présente un bit récessif alors qu'un autre présente un bit dominant perd l'arbitrage.

Arbitrage :

Exemple : trois nœuds souhaitent prendre simultanément le contrôle de la communication :

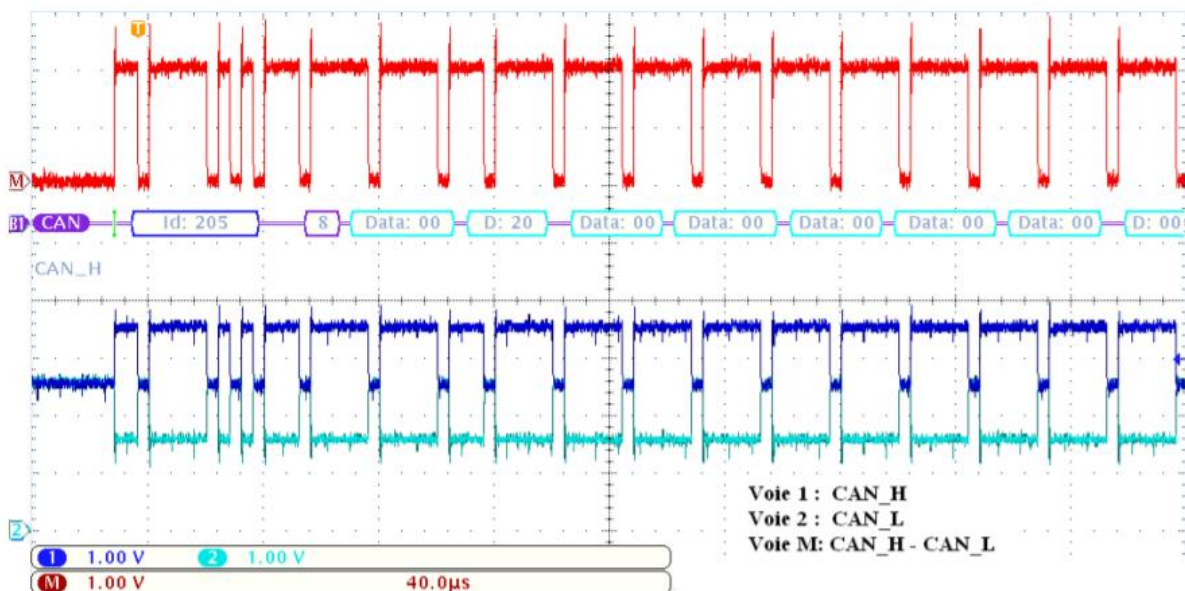


Tous les nœuds perdants deviennent automatiquement des récepteurs du message, et ne tentent à nouveau d'émettre que lorsque le bus se libère.

Pour déterminer la priorité des messages, le CAN utilise la méthode CSMA/CR (Carrier Sense, Multiple Access with Collision Resolution) avec la capacité de l'arbitrage non destructif afin d'offrir une disponibilité maximale du bus.

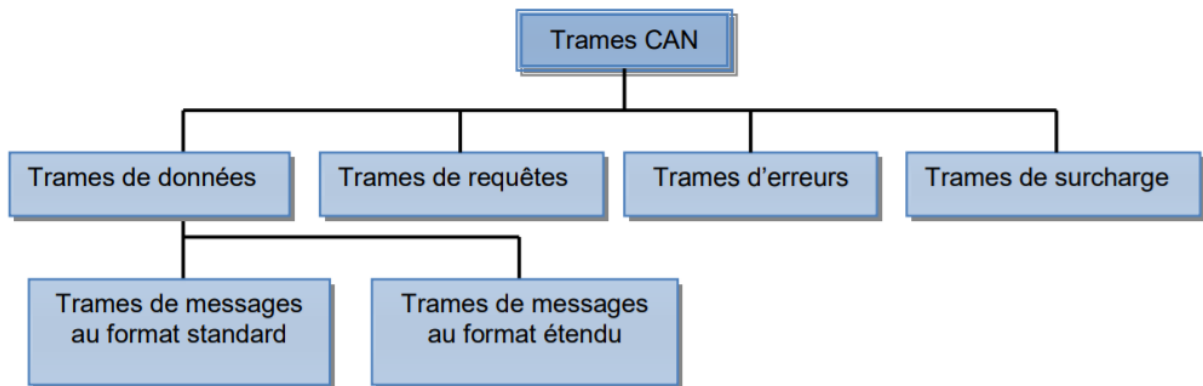
VERSION DU BUS CAN ET NIVEAUX ELECTRIQUES

BUS CAN	
BUS CAN LOW SPEED	BUS CAN HIGH SPEED
☒ Débit : 10kb/s à 125 kb/s	☒ Débit : 250kb/s à 1 Mb/s
☒ Tension d'alimentation: 5V	☒ Tension d'alimentation: 5V
☒ Courant de sortie : > 1 mA sur 2,2 k Ω	☒ Courant de sortie : 25 à 50 mA sur 60 Ω
☒ Nombre de nœuds sur le bus : 2 à 20	☒ Nombre de nœuds sur le bus : 2 à 20
☒ Niveau dominant : CAN H = 4V CAN L = 1V	☒ Niveau dominant : CAN H = 3.5V CAN L = 1.5V
☒ Niveau récessif : CAN H = 1.75V CAN L = 3.25V	☒ Niveau récessif : CAN H = 2.5V CAN L = 2.5V
☒ Caractéristique du câble : 30 pF entre les câbles de ligne	☒ Caractéristique du câble : 2 x 120 Ω



Oscillogrammes des signaux électriques CAN HIGH SPEED 250 kbits/s

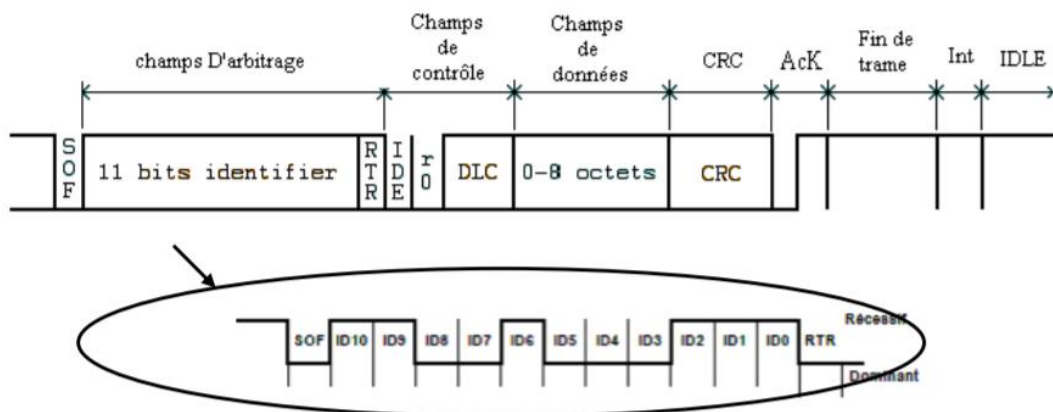
LES TRAMES DU BUS CAN



Trame de données au format standard CAN 2.0 A :



Le format CAN 2.0A est utilisé dans les voitures.



Trame de données au format étendu CAN 2.0 B :

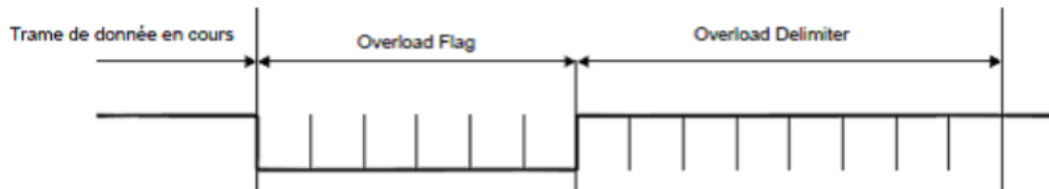


Trame de requêtes :

Une trame de requête est constituée de la même manière qu'une trame de données sauf que le champ de données est vide.

Trame de surcharge :

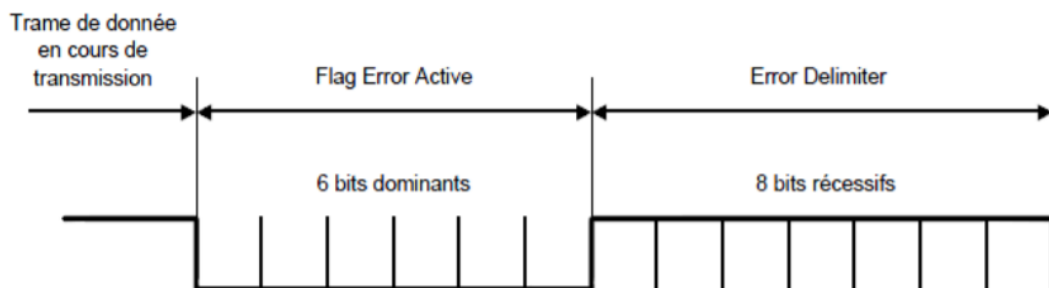
Cette trame indique qu'une station est surchargée pendant un certain laps de temps.



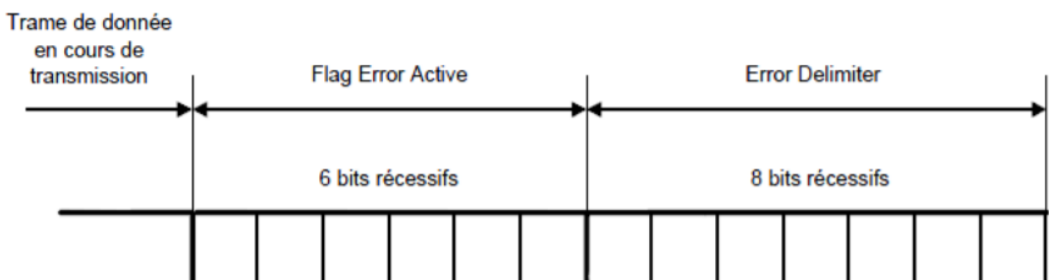
Trames d'erreurs :

Il existe deux trames d'erreur possible.

La trame d'erreur active :



La trame d'erreur passive :



CONSTITUTION D'UNE TRAME DE DONNEES

Le début de trame SOF (Start Of Frame)

Le Bit SOF Start Of Frame est dominant, il signale à toutes les stations le début d'un échange. Il ne peut apparaître que si le bus était au repos précédemment. Les nœuds se synchronisent sur le front de ce bit.

Le champ d'arbitrage ou identificateur

Le champ d'arbitrage est composé de :

- l'identificateur **ID** sur 11 bits (CAN 2.0A) ou 29 bits = 11 + 18 (CAN 2.0B) ;

- du type de trame **RTR** (Remote Transmission Request) sur 1 bit. Il est dominant pour une trame de données et récessif pour une trame de requête.

L'identificateur ID indique la fonction système réalisée par le message.

Chaque nœud recevant un message regarde si celui-ci est intéressant pour lui grâce à l'ID. Si c'est le cas, il le traite, sinon, il l'ignore.

Cet unique ID indique aussi la priorité des messages. Plus la valeur est faible, plus le message sera prioritaire. Si deux nœuds ou plus cherchent à avoir accès au bus en même temps, c'est celui de plus haute priorité qui gagne. Les messages de priorité inférieure seront automatiquement retransmis lorsque le bus sera libre.



La priorité d'un message est déterminée par la valeur de son ID : les adresses basses ont priorité sur les adresses hautes.

Pour l'identificateur les bits sont transmis dans l'ordre, de ID_10 à ID_0 (le moins significatif est ID_0). Par ailleurs les 7 bits les plus significatifs (de ID_10 à ID_4) ne doivent pas tous être récessifs. Pour des raisons de compatibilité avec des anciens circuits, les 4 derniers bits de l'identificateur (ID_3 à ID_0) ne sont pas utilisés, ce qui réduit le nombre de combinaisons possibles.

Le champ de contrôle

Il est composé de :

- la longueur des données **DLC** (Data Length Code) sur 6 bits, indique le nombre d'octets de données de 0 à 8 ;
- des bits r1 et r0 réservés pour un usage ultérieur.

Pour la trame de données, DLC indique le nombre d'octets contenu dans le champs de données. Pour la trame de requête, DLC indique le nombre d'octets contenu dans le champ de données qui devra être retourné par la trame de données demandée.

Le champ de données

Contient les données à transmettre par la trame de donnée, de 0 à 8 octets. Le bit de poids fort est transmis en premier.

Le champ de CRC

Il est composé de :

- la séquence **CRC** (Cyclic Redundancy Code) sur 15 bits ;
- du délimiteur de **CRC** (CRC Delimiter) sur 1 bit au niveau récessif.

La séquence de CRC est calculée par la procédure suivante :

Le flot de bits (hors Bit-Stuffing), constitué des bits depuis le début de la trame jusqu'à la fin du champ de données (pour une trame de données) ou bien la fin du champ de contrôle (pour une trame de requête) est interprétée comme un polynôme $f(x)$ avec des coefficients 0 et 1 affectés à la présence, effective ou non, de chaque bit.

Le polynôme obtenu est alors multiplié par x^{15} complété pour l'ajout du mot de CRC.

Le polynôme ainsi formé est divisé (modulo 2) par le polynôme générateur $g(x)=x^{15}+x^{14}+x^{10}+x^8+x^7+x^4+x^3+1$.

La chaîne de bits correspondante à ce polynôme est : $(1100010110011001)_2$.

Le reste de la division du polynôme $f(x)$ par le polynôme générateur $g(x)$ constitue la séquence CRC de 15 bits.

Le champ d'acquittement (Acknowledge)

Il est composé de :

- l'acquittement **ACK Slot**, indique qu'un nœud a reçu correctement le message et en informe le transmetteur par un bit dominant ;
- le délimiteur d'acquittement **ACK delimiter**, est un bit au niveau récessif.

Le champ de fin de trame EOF (End Of Frame)

Chaque trame est terminée par une séquence de 7 bits récessifs.

Le champ int

C'est l'espace intertrame sur 3 bits au niveau récessif. Pendant cette durée :

- aucun nœud ne peut commencer une nouvelle trame de données ou de requête ;
- seule action permise : une surcharge (overload).

IDLE

En mode inactif la ligne reste au niveau récessif.

FIABILITE ET PROTECTION DU BUS CAN

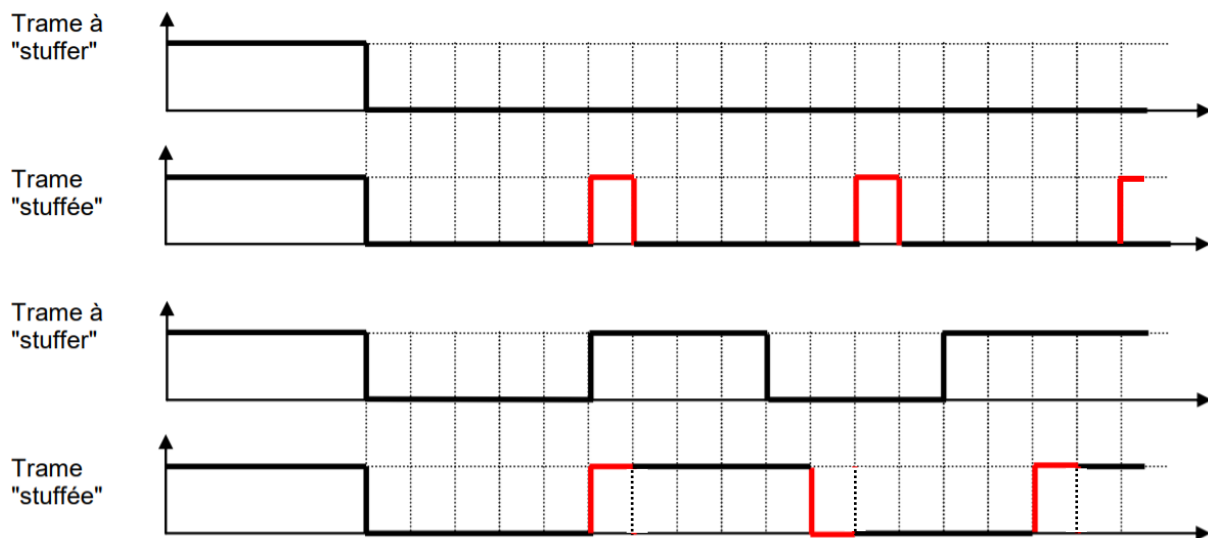
Le bit de stuffing

Le codage utilisé est de type NRZ. Cela entraîne la nécessité d'une horloge stable.

La méthode du bit-stuffing permet la resynchronisation de l'horloge lors de la réception par ajout de transition.

Dès que l'on a émis 5 bits de même polarité sur le bus, on insère un bit de polarité contraire pour casser des chaînes trop importantes de bits identiques.

Cette technique est uniquement active sur les champs de SOF, d'arbitrage, de contrôle, de CRC (délimiteur exclu). Pour un fonctionnement correct de tout le réseau, cette technique doit être implémentée aussi bien à la réception qu'à l'émission.



Exemples de trames « stuffées »



Dans le pire des cas (trame de 5 bits de même signe puis 4 bits de l'autre signe...), l'ajout du bit de « stuffing » peut augmenter le nombre de bit émis.

Les erreurs contrôlées par le bus CAN

Différentes erreurs contrôlées par le CAN

Bit error : Chaque fois qu'un émetteur envoie un bit sur le bus, il vérifie en même temps si le niveau émis sur le bus correspond à celui qu'il désire envoyer en faisant une surveillance du bus. Si le niveau ne correspond pas, il le signale par un Bit Error.

Cependant, le Bit Error n'est pas signalé dans les cas suivants :

- Aucune erreur de Bit Error n'est signalée lorsqu'un bit dominant est envoyé dans le champ d'arbitrage à la place d'un bit récessif. Le bit dominant signifie simplement une perte d'arbitrage.
- De même, pour un bit dominant lors de l'*acknowledge slot*, à la place d'un bit récessif.
- Un émetteur envoyant un *flag* d'erreur passive (bit récessif) et recevant un bit dominant, ne doit pas signaler un *Bit Error*.

Stuff error : Une erreur de *Stuffing* est détectée à chaque fois qu'il y a 6 bits ou plus consécutifs de même signe sur le bus. Une erreur de *Stuffing* ne doit être signalée que dans les champs d'identificateurs, de commande et de CRC.

CRC error : le CRC calculé par le récepteur est différent de celui envoyé par l'émetteur.

Acknowledge delimiter : le récepteur n'observe pas un bit récessif lors du champ de *Acknowledge Delimiter*. Il en est de même pour le *CRC Delimiter*.

Acknowledgement error : Une erreur de *Slot Acknowledge* est signalée par l'émetteur s'il ne lit pas un bit dominant lors du champ de slot acknowledge.

EXERCICES

Question 1

Déterminez le nombre de capteurs/actionneurs TOR (Tout Ou Rien) différents qu'un noeud peut gérer dans une seule trame de données.

.....

.....

Question 2

Dans le cas d'une trame au format CAN 2.0A standard et au format étendu CAN 2.0B, calculez la longueur mini et maxi de la trame.

.....

.....

.....

.....

Question 3

Calculez le rendement du protocole CAN lorsqu'il émet une trame de données complète (cas de la trame CAN 2.0A standard).

.....

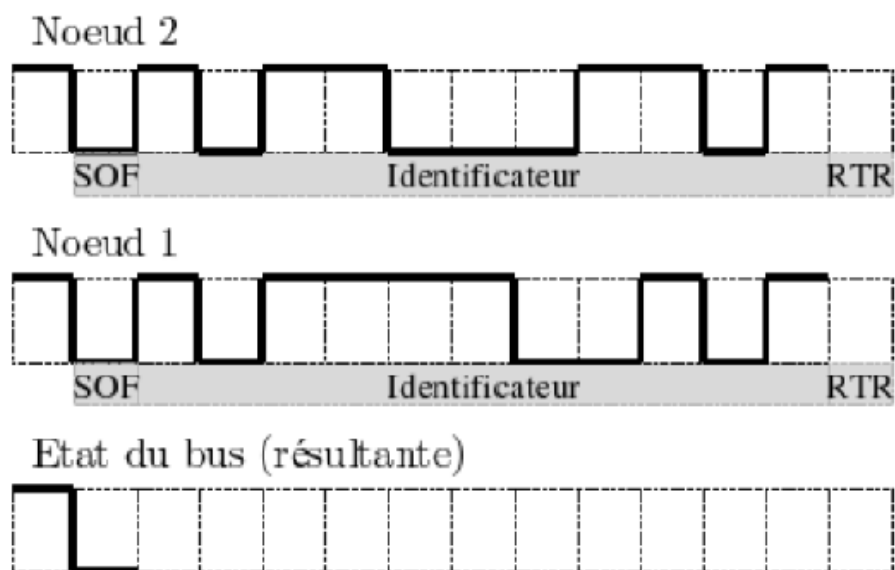
Question 4

Avec des trames de 60 bits en moyenne, calculez le nombre maximum de trames pouvant circuler sur le bus en une seconde pour un débit maximum de 1Mbit/s.

.....

Question 5

Complétez le chronogramme (résultante sur le bus : arbitrage) et indiquez le noeud qui a réussi à émettre sa trame.



Question 6

Que se passe-t-il si 2 noeuds émettent chacun une trame possédant le même identificateur mais une des 2 trames est une trame de données et l'autre une trame de requête.

Faites une conclusion sur la priorité entre une trame de données et une trame de requêtes.

.....

.....

.....

.....

.....

.....

Question 7

Complétez la trame (jusqu'au champ CRC exclu) dans le cas où un noeud émet les données 'B' (0x42) suivi de la valeur 6.



Un dispositif d'acquisition du signal sur une éolienne véhicule les informations des capteurs sur un bus CAN.

Message	Length	Data	Period	Count	RTR-Per.	RTR-Cnt.
09FD0201h	8	FF BE 00 5F F5 FA FF FF	100	9935		0
18EEFF01h	8	00 00 30 E8 00 BE A0 C0	1997	497		0



Dans la capture de trame (page précédente) d'identifiant 09FD0201h :

- les octets DATA 7 et DATA 6 représentent l'information de vitesse de vent apparent (respectivement LSB et MSB)
- les octets DATA 5 et DATA 4 représentent l'information d'angle de vent apparent (respectivement LSB et MSB).

Question 8a

Identifiez et relevez les valeurs des champs données qui contiennent la vitesse du vent.

Question 8b

Sachant que l'unité dans laquelle est codée l'information de vitesse sont des **m/min**.

À quelle valeur décimale cela correspond t-il ? et en m/s ?

Question 9

Complétez le diagramme suivant en respectant la méthode de « BitStuffing ».

Trame avant *stuffing*

[illegible]Trame avec *stuffing*[illegible]